2025 - 2026

GRUPO STUDIUM ciclos

Tema 5: Temario

JAVASCRIPT

LENGUAJES DE MARCAS Y SISTEMAS DE GESTIÓN DE INFORMACIÓN

www.grupostudium.com
informacion@grupostudium.com
954 539 952

5.1 Introducción

En el primer trimestre vimos los fundamentos de **HTML** para dar estructura y contenido a nuestras páginas webs.

También hemos estudiado **CSS** para dar una buena apariencia a nuestras páginas webs.

Ahora, nos adentraremos en el mundo de la **programación de guiones** o **scripts**, para dar mayor potencial a nuestros proyectos webs, tanto en contenido como en presentación y diseño.

JavaScript fue creado inicialmente para "hacer que las páginas web estén vivas".

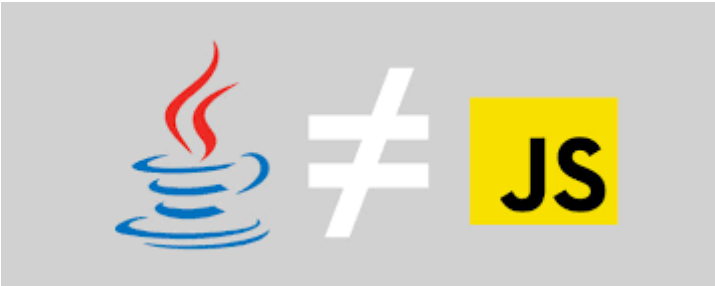
Los programas en este idioma se llaman scripts. Se pueden escribir directamente en el código HTML de una página web y ejecutarse automáticamente a medida que se carga la página.

Los scripts se proporcionan y ejecutan como texto plano. No necesitan preparación especial o compilación para ejecutarse.

En este aspecto, JavaScript es muy diferente a JAVA (que es un lenguaje que necesita ser interpretado).

Cuando se creó JavaScript, inicialmente tenía otro nombre: "LiveScript". Pero JAVA era muy popular en ese momento, por lo que se decidió que posicionar un nuevo idioma como un "hermano pequeño" de JAVA ayudaría.

Pero a medida que evolucionaba, JavaScript se convirtió en un lenguaje totalmente independiente con su propia especificación llamada **ECMAScript**, y ahora no tiene ninguna relación con JAVA.



Como **lenguaje de programación**, se define como orientado a objetos, basado en prototipos, imperativo, débilmente tipado y dinámico.

Hoy en día, JavaScript puede ejecutarse no solo en el navegador, sino también en el servidor, o en cualquier dispositivo que tenga un programa especial llamado **motor de JavaScript**.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

El navegador tiene un motor incorporado a veces llamado "máquina virtual de JavaScript".

Diferentes motores tienen diferentes "nombres de código". Por ejemplo:

- V8 - en Chrome y Opera.
- SpiderMonkey - en Firefox.
- Hay otros nombres en código como "Trident" y "Chakra" para diferentes versiones de IE, "ChakraCore" para Microsoft Edge, "Nitro" y "SquirrelFish" para Safari, etc.

Los términos anteriores son buenos para recordar porque se usan en artículos para desarrolladores en Internet. Los usaremos también. Por ejemplo, si "una característica X es compatible con V8", entonces probablemente funcione en Chrome y Opera.

Los motores son complicados., pero sus fundamentos son fáciles:

1. El motor (incrustado si es un navegador) lee ("analiza") el script.
2. Luego convierte ("compila") el script al lenguaje de máquina.
3. Y luego ejecuta el código de la máquina, bastante rápido.

El motor aplica optimizaciones en cada paso del proceso. Incluso observa el script compilado a medida que se ejecuta, analiza los datos que fluyen a través de él y aplica optimizaciones al código de la máquina basándose en ese conocimiento. Cuando está hecho, los scripts se ejecutan bastante rápido.

¿Qué puede hacer JavaScript en el navegador?

El JavaScript moderno es un lenguaje de programación "seguro". No proporciona acceso de bajo nivel a la memoria o la CPU, ya que se creó inicialmente para navegadores, los cuales no lo requieren.

Las capacidades de JavaScript dependen en gran medida del entorno en el que se esté ejecutando. Por ejemplo, Node.JS admite funciones que permiten a JavaScript leer / escribir archivos arbitrarios, realizar solicitudes de red, etc.

En el navegador JavaScript puede hacer todo lo relacionado con la manipulación de la página web, la interacción con el usuario y el servidor web.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Por ejemplo, JavaScript en el navegador es capaz de:

- Agregar un nuevo HTML a la página, cambiar el contenido existente, modificar los estilos.
- Reaccionar ante las acciones del usuario, cómo con los clics del mouse, los movimientos del puntero, las teclas presionadas.
- Enviar solicitudes a través de la red a servidores remotos, descargar y cargar archivos (las llamadas tecnologías AJAX y COMET).
- Obtener y configurar cookies, hacer preguntas al visitante, mostrar mensajes.
- Recordar los datos del lado del cliente ("almacenamiento local").

¿Qué NO PUEDE hacer JavaScript en el navegador?

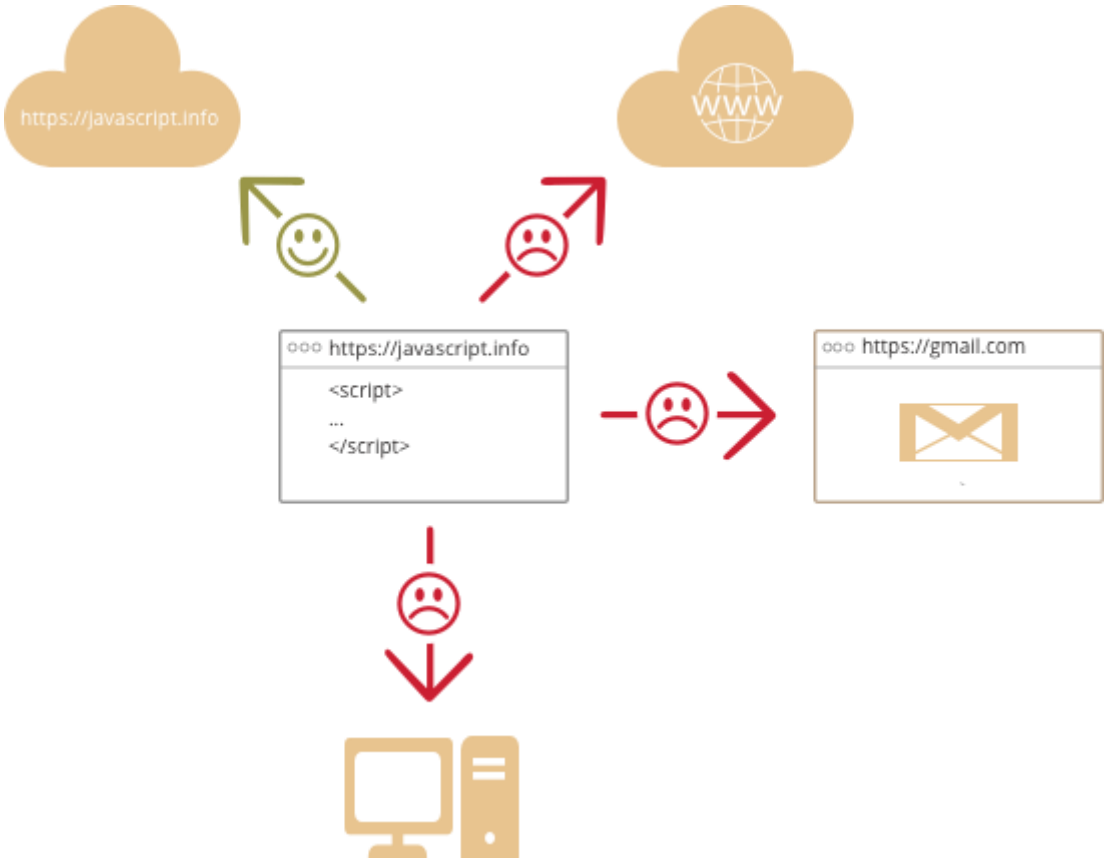
Las capacidades de JavaScript en el navegador están **limitadas** por la seguridad del usuario. El objetivo es evitar que una página web maliciosa pueda acceder a información privada o dañar los datos del usuario.

Ejemplos de tales restricciones incluyen:

- JavaScript en una página web no puede leer / escribir archivos arbitrarios en el disco duro, copiarlos o ejecutar programas. No tiene acceso directo a las funciones del sistema operativo. Los navegadores modernos le permiten trabajar con archivos, pero el acceso es limitado y solo se proporciona si el usuario realiza ciertas acciones, como "soltar" un archivo en una ventana del navegador o seleccionarlo mediante una etiqueta <input>.
- Hay formas de interactuar con la cámara / micrófono y otros dispositivos, pero requieren el permiso explícito de un usuario. Por lo tanto, una página habilitada para JavaScript no puede habilitar una cámara web, observar el entorno y enviar la información a la NSA.
- Diferentes pestañas / ventanas generalmente no se conocen entre sí. Aunque a veces sí, por ejemplo, cuando una ventana usa JavaScript para abrir la otra. Pero incluso en este caso, el JavaScript de una página no puede acceder a la otra si provienen de distintos sitios (de un dominio, protocolo o puerto diferente). Esto se llama la "Política del mismo origen". Para evitar eso, ambas páginas deben contener un código JavaScript especial que maneje el intercambio de datos. Esta limitación es, nuevamente, para la seguridad del usuario. Una página desde *http://anysite.com* que un usuario haya abierto no debe poder acceder a otra pestaña del navegador con la URL *http://gmail.com* y robar información desde allí.
- JavaScript puede comunicarse fácilmente a través de la red con el servidor de donde provenga la página actual. Pero su capacidad para recibir datos de otros sitios / dominios está paralizada. Aunque es posible, requiere un

www.grupostudium.com
informacion@grupostudium.com
954 539 952

acuerdo explícito (expresado en encabezados HTTP) desde el lado remoto. Una vez más, eso es una limitación de seguridad.



Tales límites no existen si JavaScript se usa fuera del navegador, por ejemplo, en un servidor. Los navegadores modernos también permiten complementos / extensiones que pueden solicitar permisos extendidos.

¿Qué hace que JavaScript sea único?

Hay al menos tres grandes ventajas que ofrece JavaScript:

- Integración completa con HTML / CSS.
- Las cosas simples se hacen de manera simple.
- Está soportado por todos los principales navegadores y habilitado por defecto.

JavaScript es la única tecnología de navegador que combina estas tres cosas.

Eso es lo que hace que JavaScript sea único. Por eso es la herramienta más extendida para crear interfaces de navegador.

Al planear aprender una nueva tecnología, es beneficioso revisar sus perspectivas. Así que pasemos a las tendencias modernas que lo afectan, incluidos los nuevos idiomas y las capacidades del navegador.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Y actualmente JavaScript se está convirtiendo en todo un estándar y un lenguaje de futuro, gracias a todos los **frameworks** o **entornos de trabajo** que están desarrollándose que tienen como base el JavaScript.

Variantes de JavaScript

La sintaxis de JavaScript no se adapta a las necesidades de todos. Diferentes personas quieren características diferentes.

Eso es lo que se espera, porque los proyectos y los requisitos son diferentes para cada persona.

Hace poco apareció una gran cantidad de nuevos idiomas, que se convierten a JavaScript antes de ejecutarse en el navegador.

Las herramientas modernas hacen que la transformación sea muy rápida y transparente, lo cual permite a los desarrolladores codificar en otro idioma y convertirlo automáticamente.

Ejemplos de tales lenguajes:

- CoffeeScript
- TypeScript
- Dart

Hay más. Por supuesto, incluso si usamos uno de estos lenguajes, también deberíamos conocer JavaScript para entender realmente lo que estamos haciendo.

5.2 Trabajando con JavaScript

Una vez presentado el lenguaje de programación veamos cómo trabajar con JavaScript, o lo que es lo mismo, como desarrollar y ejecutar guiones o scripts de este lenguaje tan potente y con tanto potencial de futuro (Hay quienes dicen que será el lenguaje del 99,99% de la programación dentro de pocos años).

Para trabajar con JavaScript necesitamos solamente dos elementos:

- Un entorno de edición de guiones, scripts, programas o aplicaciones
- Un navegador donde probarlos

Ya hemos trabajado en esta asignatura con **Visual Studio Code** y con **NotePad ++**. Nos sirve cualquiera de los dos, pero las herramientas extras que podemos añadir a VSC lo convierten en el editor ideal de los programadores de JavaScript, y en general, del desarrollo web.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

En cuanto al navegador, tres cuartos de lo mismo: trabajaremos con alguno de los habituales, pero tenemos que recomendar (hoy, mañana no se sabe, esto cambia casi cada día) bien **Google Chrome**, bien **Microsoft Edge** (Sí, sí, de Microsoft, se los están currando los de *Redmond*). Pero vamos, que cualquiera sirve; eso sí, necesitaremos las "Herramientas del Desarrollador" a las que se accede pulsando F12 en la mayoría de navegadores modernos y aceptables.

Clasificación de guiones

Podemos clasificar los guiones de JavaScript según dos criterios:

1. **Modo**:
    a. Automático, el script se ejecuta automáticamente sin la intervención del usuario que está interactuando con nuestra web ni de ningún otro elemento, como el sistema operativo o el propio navegador.
    b. Respondiendo a un evento del usuario, del sistema operativo, del navegador, etcétera.
2. **Ubicación**:
    a. Dentro de la propia página web (incrustados), tanto en la cabecera <head> o en el <body>. Se recomienda justo antes del final de <body>. Luego explicaremos el por qué.
    b. En fichero aparte.

Antes de comenzar viendo el primer programa en JavaScript, un par de cuestiones:

1. Los guiones incrustados se ubican entre las etiquetas <script> y </script>.
2. Todas las instrucciones de JavaScript deben acabar en punto y coma (;)

Con estas dos premisas en mente, veamos el primer ejemplo de un script automático incrustado que simplemente mostrará un mensaje en la propia página web:

El código que genera esta web sería:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>JavaScript</title>
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

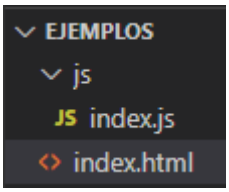
```
</head>
<body>
    <h1>JavaScript</h1>
    <script language="javascript" type="text/javascript">
        document.write("Escrito desde JavaScript");
    </script>
    <p>El resto del documento HTML ...</p>
</body>
</html>
```

index.html

La etiqueta <script> admite, entre otros, un par de atributos para indicar el tipo y el lenguaje de guion que se está escribiendo.

Por otro lado, la instrucción document.write(...); "escribe" en la web como si se tratara de cualquier etiqueta HTML para mostrar texto.

Este mismo guion, pero cuyo contenido está en un fichero externo, se montaría de la siguiente manera:



1. Creamos una carpeta llamada *js* donde colocaremos nuestros ficheros JavaScript.
2. En dicha carpeta creamos un fichero llamado index.js con la única instrucción que teníamos antes:

```
document.write("Escrito desde JavaScript");
```

3. El fichero index.html quedaría de la siguiente forma:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>JavaScript</title>
</head>
<body>
    <h1>JavaScript</h1>
    <script language="javascript" type="text/javascript" src=".\js\index.js">
    </script>
    <p>El resto del documento HTML ...</p>
</body>
</html>
```

index.html

Como podemos observar, en la etiqueta <script> hemos añadido un atributo nuevo "src" que indica la ubicación del script a insertar en ese sitio justo de la página web.

Obteniéndose, obviamente, el mismo resultado:

www.grupostudium.com
informacion@grupostudium.com
954 539 952

JavaScript

Escrito desde JavaScript

El resto del documento HTML ...

Los guiones que reaccionan a **eventos** del visitante tienen otra forma de trabajar. Estas acciones se conocen como eventos intrínsecos y serán estudiados más adelante.

5.3 Elementos fundamentales de JavaScript

A continuación, vamos a estudiar los elementos fundamentales y básicos para aprender este lenguaje de programación. Comenzaremos por los más sencillos y básico e iremos aumentando la dificultad con cada nuevo elemento. Indicar, antes de proseguir, que JavaScript no es un lenguaje complicado de aprender, al menos las bases, que es de lo que se trata en este tema. El alumnado matriculado en Desarrollo de Aplicaciones Web profundizará mucho más en este potente lenguaje el próximo curso.

Comentarios

En todo lenguaje de programación es muy importante añadir **comentarios** al código que escribimos, bien para que otras personas "entiendan" mejor el código, bien para nosotros mismos más adelante cuando repasemos el código y tengamos que hacer alguna modificación bien para corregir algo, bien para aumentar la funcionalidad del código ya elaborado.

Existen dos tipos de comentarios en JavaScript: los que ocupan una sola línea y los que ocupan más de una.

Los que ocupan una línea, deben empezar por **doble barra** o **inclinadas** hacia la derecha:

```
// Comentario de una línea
document.write("Escrito desde JavaScript"); // Mostrar por pantalla
```

En el fragmento anterior tenemos dos comentarios, el primero que ocupa toda la línea desde el principio, y un segundo comentario que está justo detrás de la instrucción a ejecutar. Todo lo que hay detrás de las dos barras se considera comentario, no se ejecuta nada. Y este tipo de comentario, como vemos, puede comenzar en cualquier parte de la línea, pero a partir de las dos barras, ya todo es comentario. Si queremos meter una instrucción nueva, será ya en la siguiente línea.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Los otros comentarios comienzan por **barra y asterisco** y acaban en **asterisco barra**:

```
/* Comentario
en
más
de
una
línea */
document.write("Escrito desde JavaScript"); /* Mostrar por pantalla */
```

En este caso el primer comentario ocupa claramente varias líneas y el segundo también puede comportarse como el "//". Pero en este tipo de comentario todo lo que haya entre "/*" y "*/" se considerará comentario, esté o no en la misma línea o en cientos (Procurad no abusar de los comentarios tampoco, ni mucho, ni poco, lo justo).

Variables

La parte más básica de cualquier lenguaje de programación son las **variables** que se pueden manejar, en las cuales se guardarán valores necesarios para la ejecución de los programas.

En JavaScript se pueden declarar las variables de dos formas: la primera sería simplemente escribiendo la variable y asignarle un valor (declaración implícita) y la segunda sería usando la palabra reservada let seguida del nombre de la variable (declaración explícita). Se recomienda esta segunda forma. En programas antiguos se puede ver también var en lugar de let.

Ejemplos:

```
// Forma actual
let numero1;
// Formas antiguas
numero2 = 7;
var numero3 = 7;
```

Como nombre de variables podemos usar cualquier palabra que no coincida con las **palabras reservadas** del propio lenguaje. También se debe cumplir que empiece por letra o subrayado o un signo de dólar. No se admiten ni espacios en blanco ni tildes ni ñ, para evitar problemas de compatibilidad. Debemos ser lo más estándares posibles a la hora de crear nuestros proyectos web.

JavaScript **diferencia entre mayúsculas y minúsculas**, con lo que la variable "numero" es diferente a la variable "Numero" o a "NUMERO".

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Como norma general, usaremos **nombres descriptivos** de las variables, que nos digan qué contienen y la nomenclatura Camel sería lo ideal. La primera palabra en minúsculas y las siguientes con la primera en mayúsculas: fechaNacimientoCliente.

Ámbito de las variables

Las variables cuando se definen tienen un **ámbito** o zona donde son reconocidas. Fuera de ese ámbito, la variable no existe. Por ejemplo, si definimos una variable dentro de una función, y le damos valor, dicha variable solamente tiene sentido y contenido dentro de la función donde fue definida. Las llaves { } se usan como los límites de los ámbitos.

Ejemplo:

```
function funcion1()
{
    let variable1 = 7;
}
function funcion2()
{
    document.write(variable1);
}
```

Al ejecutar este código y llamar a funcion2(), daría un error, pues "variable1" solamente tiene sentido, y por tanto contenido, en el ámbito de la función funcion1().

Existen también variables globales que tiene validez en todo el código, pero su uso está totalmente desaconsejado por los problemas que puede ocasionar, pues es más difícil controlar su contenido.

Tipos de datos

Si nos fijamos al declarar una variable, no se le asigna un **tipo**, es decir, se le puede asignar cualquier valor. Y a lo largo del programa puede tener diferentes valores y diferentes tipos. Esto puede llevar a errores difíciles de encontrar, pero con una buena metodología a la hora de nombrar y usar las variables, no debemos tener mayores problemas.

Existen unos tipos básicos en JavaScript:

- Cadenas de texto o string
- Valores numéricos o Nnmber
- Booleanos o boolean
- Objetos u object
- Nulos o null
- Indefinidos o undefined

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Con la función typeof() podemos averiguar el tipo del contenido de una variable.

Veamos un ejemplo...

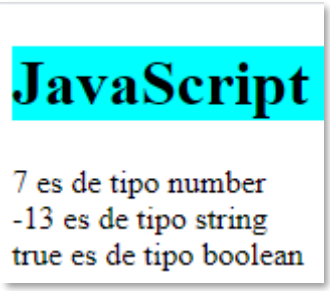
```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="./css/estilos.css">
    <title>JavaScript</title>
</head>
<body>
    <h1>JavaScript</h1>
    <script language="javascript" type="text/javascript" src=".\js\index.js">
    </script>
</body>
</html>
```

index.html

```
let variable1 = 7;
let variable2 = "-13";
let variable3 = true;

document.write(variable1+" es de tipo "+typeof(variable1));
document.write("<br/>");
document.write(variable2+" es de tipo "+typeof(variable2));
document.write("<br/>");
document.write(variable3+" es de tipo "+typeof(variable3));
```

index.js

Y en el navegador...



Cadenas

Cuando asignamos a una variable un **texto**, esta se convierte en el tipo string o cadena.

Las cadenas siempre se denotan por estar entre comillas dobles o simples: “Hola”, ‘Hola’.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Dentro de una cadena delimitada por comillas dobles podemos incluir comillas simples y viceversa: “Esta ‘palabra’ está dentro de una cadena”.

Dentro de las cadenas podemos incluir una serie de caracteres especiales que no se podrían representar de ningún otro modo.

Carácter	Significado
\n	Salto de línea
\r	Retorno de carro
\f	Salto de página
\t	Tabulación
****	El propio carácter \
\’	La propia comilla simple
\”	La propia comilla doble
\b	Carácter anterior

Si queremos representar una ruta no podemos poner “C:\Documentos\fichero.doc” dentro de una cadena. Habría que hacerlo de la siguiente forma:

```
let cadena = "C:\\Documentos\\fichero.doc";
```

Las cadenas se pueden concatenar con el signo de más (+):

```
let cadena1 = "Hola";
let cadena2 = " Javier";
document.write(cadena1+cadena2);
```

Y obtendremos:

Valores numéricos

En JavaScript podemos usar multitud de representaciones numéricas:

```
let numero1 = 12345; // Número entero
let numero2 = -382; // Número negativo
let numero3 = 3.1415; // Número decimal
let numero4 = 6.023E+23; // Número potencia de 10
```

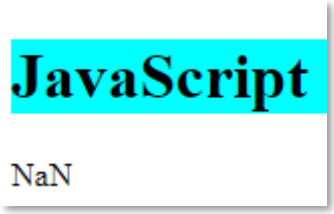
Existen un par de valores numéricos especiales. **NaN** para denotar que dicho valor “No es un Número” (Not a Number) e **Infinity** para indicar el valor numérico de infinito que se obtiene en ciertas operaciones matemáticas.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Probad lo siguiente:

```
let numero1 = 3;
let numero2 = "Hola";
document.write(numero1 / numero2);
```

Obteniendo:



Y también:

```
let numero1 = 3;
let numero2 = 0;
document.write(numero1 / numero2);
```

En el navegador:

Booleanos

En JavaScript disponemos de dos valores booleanos como en casi cualquier lenguaje de programación. Se trata de **true** y de **false**. Fundamentalmente sirven para evaluar condiciones, como veremos más adelante.

Variables nulas

Cuando una variable no contiene nada su contenido es nulo o null. También podemos hacer que una variable se vacíe asignándole el valor null y así liberamos el espacio de memoria que ocupaba con anterioridad.

Conversión de tipos

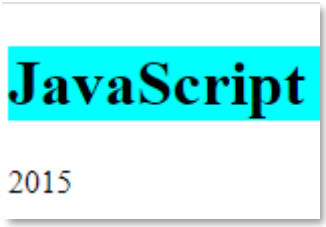
Cuando en una misma operación mezclamos variables con tipos diferentes, si entre ellos existe una cadena, éste será el tipo del resultado.

Ejemplo:

```
let variable1 = 20;
let variable2 = "15";
document.write(variable1 + variable2);
```

Se obtiene la cadena “2015” y no 35 como podría pensarse.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

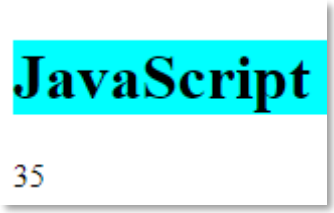


Si por el contrario queremos forzar a que variable1 se tome como número debemos usar funciones de conversión: parseInt() para convertir en número entero, y parseFloat() para convertir en número decimal.

Ahora:

```
let variable1 = 20;
let variable2 = "15";
document.write(variable1 + parseInt(variable2));
```

Ahora sí, obtendríamos el valor numérico de 35:



Con la función toString() obtendríamos el efecto contrario: pasaríamos cualquier variable numérica a cadena, aunque, como hemos visto, si concatenamos cualquier tipo con cadena, todo se vuelve cadena.

Funciones

Cuando escribimos un programa, en muchas ocasiones tendremos que repetir varias veces las mismas instrucciones dentro del código; estas tareas pueden ser exactamente iguales o con alguna variación. En cualquier caso, sería bueno aprovechar el código que ya hemos escrito una vez para reutilizarlo todas las veces necesarias. Las **funciones** nos permiten realizar esto. Consisten en una agrupación de instrucciones con un nombre propio y unos parámetros que podemos enviar para personalizar cada ejecución y que no tengan siempre el mismo comportamiento.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Para definir una función en JavaScript se emplea la palabra reservada *function* seguida de un identificador que obedece a las mismas reglas que los nombres de las variables. Este nombre sirve para identificar el fragmento de código que constituye la función y así poder llamarlo posteriormente para "usarla". Tras el nombre aparecen unos paréntesis donde se incluirán todas las variables que admite la función para operar. Estas variables se denominan parámetros. Tras el cierre del paréntesis de los parámetros se escribe todo el código de la función englobado entre llaves.

El número de parámetros que admite una función es ilimitado.

Una vez definida esta función, se puede llamar tantas veces como sea necesaria y con diferentes parámetros, con lo que obtendremos con la misma función resultados diferentes, pero sin escribir más código.

Veamos un ejemplo donde se crea una función muy simple llamada **funcion()** que recibe dos parámetros calculando la suma de dichos elementos si son numéricos o la concatenación si alguno es una cadena. La definición, en el fichero index.js, sería:

```
function funcion(var1, var2)
{
    document.write(var1 + var2);
}
funcion(1,3);
document.write("<br/>");
funcion(1,"3");
```

Al ejecutar en el navegador:

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Para ciertas funciones, se debe devolver un valor. Para ello usaremos la palabra reservada *return*.

Ejemplo:

```
function funcion(var1, var2)
{
    document.write(suma(var1, var2));
}
function suma(v1, v2)
{
    return (v1 + v2);
}
funcion(1,3);
document.write("<br/>");
funcion(1,"3");
```

Aquí estamos usando una nueva función llamada **suma** que recibe dos parámetros, los suma y los devuelve al lugar donde fue llamada.

En el navegador obtenemos lo mismo que en el ejemplo anterior.

Operadores y funciones matemáticas

Uno de los principales usos de un lenguaje de programación es el realizar cálculos, comparaciones y relaciones entre diferentes datos, lo que se denomina con carácter general 'Operaciones'. JavaScript proporciona al programador una gran variedad de operadores que estudiaremos a continuación.

Las funciones matemáticas más comunes no se ofrecen de manera independiente, sino que se exponen como métodos de un objeto global especial llamado **Math** que usaremos en este tema.

Operadores

Se muestran a continuación los operadores agrupados por significados.

Aritméticos	
Cambio de signo	-
Incremento	++
Decremento	--
Multiplicación	*
División	/
Suma	+
Resta	-
Módulo	%

Si al evaluar una expresión matemática alguno de los operandos es nulo, el resultado será siempre nulo también.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Lógicos	
AND	&&
OR	||
NOT	!

Los operadores lógicos se usan a la hora de evaluar expresiones lógicas para tomar decisiones, como veremos en un apartado posterior.

Relacionales	
Igual a	==
Distinto de	!=
Mayor	>
Mayor o igual	>=
Menor	<
Menor o igual	<=
Igualdad estricta	===
Desigualdad estricta	!==

Los operadores relaciones sirven para comparar valores o expresiones, sobre todo para evaluar condiciones.

Asignación	
Asignación directa	=
Con adición	+=
Con sustracción	-=
Con multiplicación	*=
Con división	/=
Con módulo	%=

Por último, los operadores de asignación se usan para dar o cambiar valores a las variables.

Funciones matemáticas

En este aparado presentamos las principales funciones matemáticas de las que disponemos para todo tipo de cálculos.

Logaritmo:

```
Math.log(variable);
```

Exponente:

```
Math.exp(variable);
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

Raíces cuadradas:

```
Math.sqrt(variable);
```

Potencias:

```
Math.pow(base, exponente);
```

Valor absoluto:

```
Math.abs(variable);
```

Parte Entera inferior:

```
Math.floor(variable);
```

Parte entera superior:

```
Math.ceil(variable);
```

Redondeo:

```
Math.round(variable);
```

Números aleatorios:

```
Math.random();
```

Esta función realmente obtiene valores entre 0 y 1. Para obtener números aleatorios entre otros valores, por ejemplo, entre 5 y 15, debemos hacer algo tal que así:

```
Math.round(15 - 5) * Math.random() + 5;
```

Funciones trigonométricas:

```
Math.sin(variable);
Math.cos(variable);
Math.tan(variable);
Math.acos(variable);
Math.asin(variable);
Math.atan(variable);
Math.atan2(variable);
```

Máximo:

```
Math.max(variable1, variable2);
```

Mínimo:

```
Math.min(variable1, variable2);
```

Número E:

```
Math.E;
```

Número PI:

```
Math.PI;
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

5.4 Estructuras de control de flujo

Las **estructuras de control** de un lenguaje de programación lo dotan de flexibilidad y proporcionan a los programadores la capacidad de decisión y de realizar procesos de manera repetitiva. Con ellos seremos capaces de dirigir nuestro código a través de distintas opciones en función de ciertas condiciones, o conseguiremos que determinadas porciones de código se repitan varias veces.

Lo que define a un **lenguaje de programación** son la capacidad de guardar información en variables, y la capacidad de especificar el flujo que deben tomar dichos datos mediante las estructuras de control. Sin esos elementos básicos no podríamos decir que se trata de un lenguaje de programación. El lenguaje HTML, por ejemplo, al carecer de estos elementos, no se considera lenguaje de programación; de hecho, es un lenguaje de marcado, como se ha visto al principio de esta asignatura.

Nos disponemos ahora a conocer estas estructuras en nuestro querido JavaScript.

Condicionales

Entre las instrucciones condicionales nos encontramos con la estructura **if-else** y la **switch-case**.

La estructura de decisión **if-else** es, sin duda, la que más veces se utiliza en el código de un programa, independientemente del lenguaje que se use.

Permite tomar decisiones en función de unas determinadas condiciones que se impongan para ejecutar uno u otro código según lo que indiquen dichas condiciones. De esta forma se puede dotar de cierta "inteligencia" a nuestros programas, considerando como tal el hecho de que podrá tomar sus propias decisiones basándose en la situación que se dé en cada momento.

La sintaxis más básica sería:

```
if(condición)
{
    Instrucciones a ejecutar si la condición es verdadera
}
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

Podemos añadir un segundo nivel para indicar las instrucciones que se deberían ejecutar en caso de que la condición fuera falsa:

```
if(condición)
{
    Instrucciones a ejecutar si la condición es verdadera
}
else
{
    Instrucciones a ejecutar si la condición es falsa
}
```

La condición no tiene por qué ser atómica, y se pueden encadenar condiciones con los operadores lógicos como AND (&&), OR (||) o NOT (!).

Por último, indicar que podemos anidar estructuras **if-else** tantas veces como sea necesario, pero sin pasarse, pues podemos llegar a producir los llamados "código espaguetis":

```
if(condición1)
{
    if(condición2)
    {
        Instrucciones a ejecutar si la condición2 es verdadera
    }
    else
    {
        Instrucciones a ejecutar si la condición2 es falsa
    }
}
else
{
    Instrucciones a ejecutar si la condición1 es falsa
}
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

Para evitar muchos anidamientos de estructuras if-else, en algunas ocasiones, podemos sustituirlos por una única estructura **switch-case**, que tiene la siguiente sintaxis:

```
switch(expresión)
{
    case valor1:
    case valor2:
        Código caso 1 y 2
        break;
    case valor3:
        Código caso 3
        break;
    …
    default:
        Código caso por defecto
}
```

La explicación de esta estructura es bien sencilla. Evaluamos la *expresión* indicada junto al *switch* y se ejecuta el código asociado al valor que coincida de los *case* disponibles. Y si la *expresión* no es igual a ninguno de los *case*, se ejecutan las instrucciones del bloque *default*.

Las instrucciones *break* son necesarias para que la ejecución de las sentencias se vaya al final del bloque *switch*, y no continúe por los *case* siguientes. Pero si queremos, por ejemplo, que, al coincidir la *expresión* con el valor de un *case*, también ejecute las sentencias siguientes, en ese *case* no pondremos el *break*.

Esta estructura **switch-case** solamente permite la igualdad entre la expresión y los diferentes calores de los case. No admite otro tipo de comparación como menor o igual, menor, ni ninguno similar. Solo igualdad.

Bucles

Tenemos **for**, **for-in**, **while** y **do-while**. Un bucle es aquel conjunto de instrucciones que se repite un cierto número de veces, hasta que se completa un número de vueltas determinado o bien se deja de cumplir cierta condición o grupo de condiciones.

Cuando conocemos de antemano el número de vueltas que vamos a dar en el bucle, usaremos la estructura **for**. Si, por el contrario, no conocemos el número devueltas que tenemos que dar, usaremos la estructura **while** en alguna de sus dos versiones como veremos más adelante.

La estructura **for** en su primera versión usa un iterador (una variable) que toma un valor inicial y va aumentando su valor progresivamente hasta llegar a un valor final. El valor inicial, el valor final y el incremento de dicho iterador definen el número de

www.grupostudium.com
informacion@grupostudium.com
954 539 952

veces que se harán las instrucciones contenidas en el bloque del bucle. El siguiente ejemplo muestra por pantalla el propio iterador, del 0 al 10:

```
for(i=0;i<=10;i++)
{
    Instrucción a repetir
}
```

La expresión i=0 indica el valor inicial que se le asigna al iterador i. la expresión i<=10 indica la condición que debe cumplirse para que se sigan dando vueltas en el bucle. La expresión i++ indica cómo debe aumentar el valor del iterador. Lo habitual es que sea de uno en uno, que lo podemos expresar con el ++, pero se puede aumentar o disminuir en la cantidad que se quiera.

Dentro del bloque del bucle podemos insertar tantas instrucciones como necesitemos. Incluso podemos incluir otro bucle anidado, pero en este caso hay que tener especial cuidado en no usar el mismo iterador.

Existe una variante del **for** que consiste en ejecutar el cuerpo del bucle tantas veces como miembros tenga una colección de objetos o una matriz. Más adelante, en el apartado de Programación Orientada a Objetos entenderemos mejor estos tipos de elementos.

Aquí simplemente veremos su funcionamiento. Imaginemos que tenemos un objeto llamado "Pais" en cuyo contenido tenemos un atributo llamado "Provincias" que devuelve una colección de las provincias españolas. Si ejecutamos el siguiente código:

```
var prov = "";
for(prov in Pais.Provincias)
{
    document.write(prov);
}
```

Nos irán apareciendo todas y cada una de las provincias de la geografía española.

El bucle **while** tiene la siguiente sintaxis:

```
while(condición)
{
    Instrucciones a ejecutar
}
```

Este bucle está ejecutando las instrucciones en su interior un número indeterminado de veces. El bucle acabará de ejecutarse cuando la condición deje de ser verdadera. Obviamente, entre esas instrucciones debe darse algún caso en el que haga que dicha condición deje de ser verdadera, o estaremos ante un bucle infinito, que nunca acaba.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Ejemplo:

```
var i = 0;
while(i<=10)
{
    document.write(i);
    i++;
}
```

Este bucle es similar al visto con anterioridad con el for. En muchas ocasiones, hay bucles **for** y bucles **while** similares o iguales. En otras ocasiones no se pueden encontrar equivalencias.

También existe una variación al bucle **while**. Pensemos en qué ocurre si la primera vez que evaluamos la condición del **while**, esta no es cierta. Pues que no se entra nunca en el bucle. Pero en ocasiones necesitamos que, al menos, una vez se ejecute el contenido del bucle. En dicho caso debemos utilizar la variante **do-while** tal como se muestra a continuación:

```
do
{
    Instrucciones a ejecutar
} while(condición);
```

Como en este caso la condición se evalúa al final, al menos una vez se van a ejecutar las instrucciones del interior del bloque delimitado por las llaves.

NOTA: En la versión **while**, tras la condición no se coloca nada. En la versión **do-while**, tras la condición hay que cerrar con punto y coma (;).

Existe un par de instrucciones relacionadas con el flujo normal de la ejecución de un programa que son **break** y **continue**, pero está totalmente **desaconsejado** su uso.

Por ejemplo, el **break** dentro de un bucle hace que este se acabe repentinamente, mientras que el **continue** salta la ejecución de una iteración y sigue con la ejecución del bucle. El siguiente código incluye un bucle **for** que se debería ejecutar 10 veces, pero al llegar a la tercera vuelta, forzamos la terminación del bucle con un **break**:

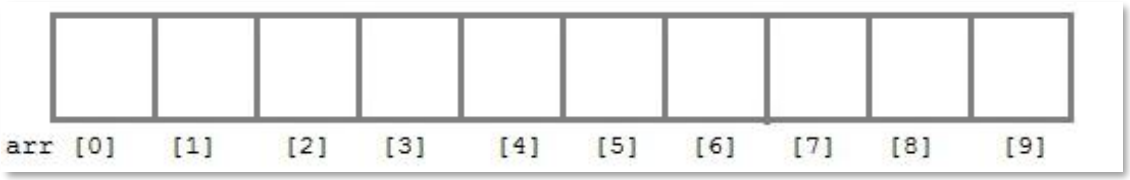
```
for(i=0;i<=10;i++)
{
    if(i==3)
    {
        break;
    }
    document.write(i);
}
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

5.5 Matrices

Una **matriz** es un conjunto de datos colocados de forma adyacente en la memoria de manera que nos podemos referir a ellos con un solo nombre común y una posición.

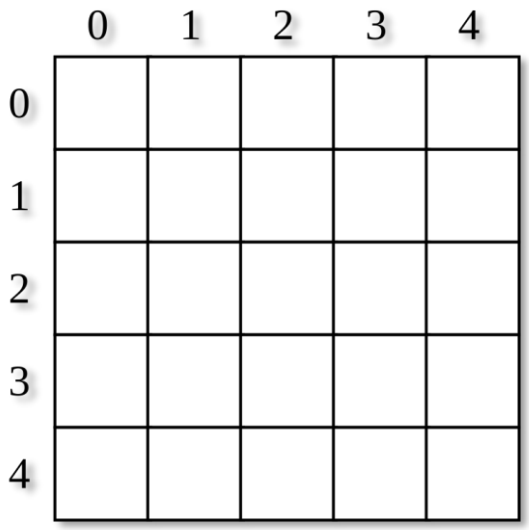
Es una forma rápida y cómoda de guardar muchos datos, en vez de usar muchas variables.



En la imagen anterior podemos ver una matriz, también llamadas tablas y arrays, de tamaño 10 y llamada *arr*. La primera posición corresponde a *arr*[0] y la última *arr*[9].

Las **matrices** en JavaScript son dinámicas, eso significa que su tamaño puede variar a lo largo de la ejecución del programa para adaptarse a las necesidades.

También podemos tener matrices con varias dimensiones. Por ejemplo, una tabla de dos dimensiones de 5x5 tendría una presentación gráfica así:



Y podemos usar tantas dimensiones como haga falta.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Podemos crear matrices de tres maneras diferentes: creándolas sin tamaño predefinido, con tamaño definido y creándolas y dándoles valores a la vez. Veamos cada una de ellas:

```
let tabla1 = new Array();
let tabla2 = new Array(5);
let tabla3 = new Array(3,5,3,6);
```

Independientemente de cómo sean definidas, en la mayoría de los casos, tras la creación, debemos recorrer todos los elementos de la matriz, bien para rellenarlos de valores, para ver qué valores tienen, etc. Ese recorrido lo realizaremos con una estructura **for** de la siguiente forma:

```
var matriz = new Array(2,4,6,8,10);
var i;
for(i=0;i<5;i++)
{
    document.write(matriz[i]);
}
```

5.6 Cadenas de texto

Disponer de potentes funciones para el tratamiento de las cadenas de caracteres es primordial en un lenguaje de Script, ya que continuamente se deben realizar tareas de mantenimiento y verificación que implican tratar con este tipo de datos.

Estas tareas abarcan desde la validación de los datos introducidos en un formulario, hasta el análisis de frases, pasando por operaciones como proporcionar el formato correcto a un dato antes de añadirlo a una base de datos.

JavaScript proporciona un conjunto valioso de funciones que permiten al programador efectuar todo esto a través del objeto **String**.

Recordemos que una característica fundamental de las cadenas es que van ente comillas simples o dobles. También sabemos concatenar cadenas con el signo más (+).

Tenemos varias formas de crear una cadena:

```
let cadena1 = "";
let cadena2 = "Hola";
let cadena3 = new String();
let cadena4 = new String("Hola");
```

Para el tratamiento de las cadenas podemos usar multitud de funciones ya creadas que nos dan mucho potencial para el trabajo con cadenas. Pero también podemos considerar a las cadenas como matrices de caracteres, con lo que aumentamos aún más las posibilidades.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

A continuación, se muestra una tabla con las principales funciones de las cadenas que pone a nuestra disposición la última versión de JavaScript.

Funciones de Cadenas	
charAt(posicion)	Devuelve el carácter de la cadena de la posición indicada. *"abc".charAt(2) →'c'*
charCodeAt(posicion)	Devuelve el código del carácter de la cadena de la posición indicada. *"abc".charCodeAt(2) → 99*
fromCharCode(arg1, arg2, ...)	Toma como argumentos uno o más números separados por coma, y devuelve una cadena de texto con los caracteres Unicode correspondientes. *String.fromCharCode(97, 98, 99) → "abc"*
indexOf(cadena)	Devuelve la primera posición donde encuentra la cadena. *"abc".indexOf(b) → 1*
lastIndexOf(cadena)	Devuelve la última posición donde encuentra la cadena. *"abcb".lastIndexOf(b) → 3*
lenght	Nos devuelve la longitud de la cadena. *"abc".lenght → 3*
slice(posiciónInicial, posiciónFinal)	Devuelve una subcadena comenzando en posiciónInicial y acabando en posiciónFinal. *"abcde".slice(2, 4) → "cde"*
split(caracter)	Trocea la cadena original según el carácter indicado. *"a,b,c,d".split(",") →Devuelve un array con los valores a, b, c, d en cada una de sus posiciones*
substring(posiciónInicial, posiciónFinal)	Devuelve una subcadena comenzando en posiciónInicial y acabando en posiciónFinal. *"abcde".substring(2, 3) → "cd"*
toLowerCase()	Convierte la cadena en minúsculas. *"aBc".toLowerCase() → "abc"*
toUpperCase()	Convierte la cadena en mayúsculas. "aBc".toUpperCase() → "ABC"

www.grupostudium.com
informacion@grupostudium.com
954 539 952

5.7 Fechas y Horas

En multitud de proyectos web, el trabajo con fechas y horas es fundamental, bien para controlar entradas y/o salidas, bien para realizar cálculos que implican días, horas, minutos, etc.

Todas las funciones relacionadas con unidades temporales se ofrecen como métodos del objeto **Date**.

Una variable de tipo Date, que contenga una fecha, lo que realmente tiene en su interior es un valor numérico que expresa el número de milisegundos transcurridos desde el 01/01/1970 a las 00:00:00 horas[1].

Para obtener la fecha y la hora actuales simplemente definimos una nueva instancia de Date:

```
let ahora = new Date();
```

Si mostramos en un **document.write())** la variable *ahora*, obtendremos algo tal que así:

Wed Aug 25 2021 19:22:31 GMT+0200 (hora de verano de Europa central)

También se puede crear una fecha con valores determinados:

```
let fecha = new Date(2000,4,23,21,0);
```

Nos daría la fecha de 23 de mayo de 2000 a las 21:00.

Ahora se presenta un cuadro en el que se muestran las funciones que podemos aplicar a una variable de tipo Date, para tratar fundamentalmente los datos de la fecha por separados.

La forma de trabajar con ellas es siempre igual:

```
variableFecha.funcion();
```

[1] En algunos sistemas la fecha de referencia es 01/01/1900

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Funciones get de Date	
getDate()	Obtiene el día del mes dentro de la fecha dada
getDay()	Indica el número del día de la semana, siendo el 0 el domingo
getFullYear()	Devuelve el año de la fecha dada con cuatro cifras
getHours()	Obtiene la hora de una fecha dada
getMilliseconds()	Devuelve los milisegundos de una fecha dada
getMinutes()	Obtiene los minutos de una fecha dada
getMonth()	Devuelve el mes en formato número, siendo el 0 para enero
getSeconds()	Obtiene los segundos de una fecha dada
getTime()	Obtiene la hora completa, con horas, minutos y segundos de la fecha dada
getYear()	Obtiene el año de una fecha dada. Si el año está entre 1900 y 2000, el resultado es de dos cifras. En otras fechas, devuelve el año con formato de cuatro cifras.

Para la **Hora Universal Coordinada** o **UTC**, tenemos los mismos métodos, colocando UTC tras la partícula get: *getUTCdate()*, *getUTCDay()*, ...

También podemos establecer nosotros valores para fechas dadas, como cambiar el año, la hora, etc. Son los métodos set:

Funciones set de Date	
setFullYear(año)	Establece el año de la fecha dada con cuatro cifras
setHours(hora)	Establece la hora de una fecha dada
setMilliseconds(milisegundos)	Establece los milisegundos de una fecha dada
setMinutes(minutos)	Establece los minutos de una fecha dada
setMonth(mes)	Establece el mes en formato número, siendo el 0 para enero
setSeconds(segundos)	Establece los segundos de una fecha dada
setTime(hora, minutos, segundos)	Establece la hora completa, con horas, minutos y segundos de la fecha dada
setYear(año)	Establece el año de una fecha dada

E igualmente, tenemos los métodos set para las UTC.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

5.8 Interacciones con los usuarios

Una **página web** puede ser una simple presentación de información que el usuario se limita a leer simplemente, o toda una **aplicación web** en la que la interacción del usuario es mucho más intensa y en la que se realizan muchas más acciones aparte de simplemente leer: meter información, obtener resultados, pulsar botones o elegir opciones, ...Incluso podemos realizar un **juego** como una aplicación web. ¿Hay algo más interactivo que un juego en la que el usuario/jugador debe estar constantemente interactuando con la aplicación?

En este apartado no llegaremos a tanto, pero sí que presentaremos tres controles que, de forma muy sencilla y sin necesidad de recurrir a formularios, nos van a permitir interactuar con el usuario para mostrarle información y solicitar datos o confirmación de acciones.

Comenzamos por el más simple, la función alert() que muestra una ventana de información en nuestro navegador y hace una pausa hasta pulsar el botón de "Aceptar[2]" que aparece.

En este primer ejemplo, nada más cargar la web aparecerá un mensaje de bienvenida:

```html
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="./css/estilos.css">
    <title>JavaScript</title>
</head>
<body>
    <h1>JavaScript</h1>
    <script language="javascript" type="text/javascript" src=".\js\index.js">
    </script>
</body>
</html>
```

index.html

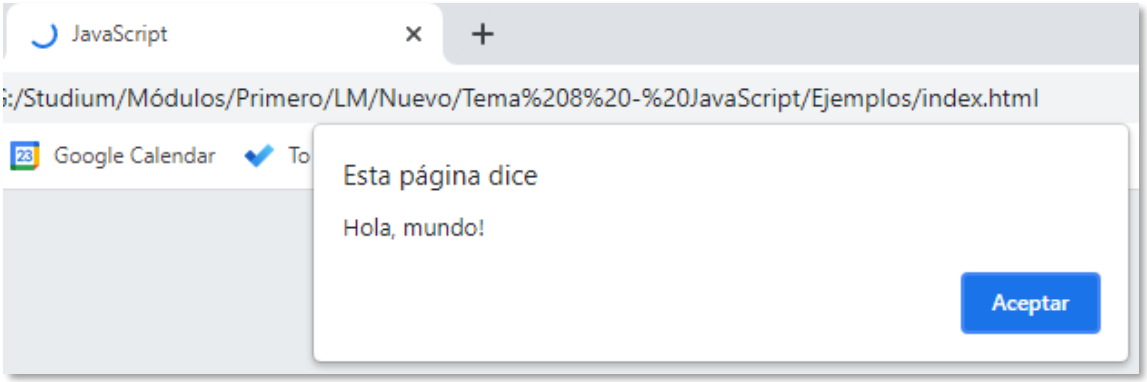
```javascript
alert("Hola, mundo!");
```

index.js

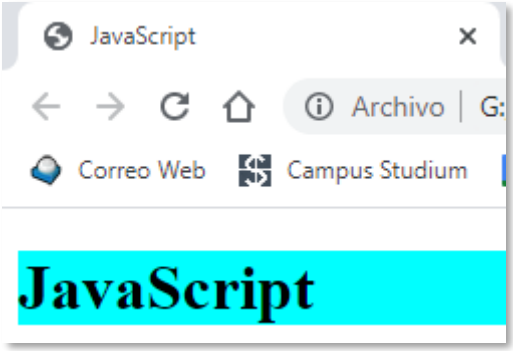
[2] Este valor lo establece el navegador; en algunos saldrá "Aceptar", en otros "OK"

www.grupostudium.com
informacion@grupostudium.com
954 539 952

En el navegador, primero...



Y al pulsar el botón de “Aceptar” ...



La mini-ventana con el mensaje se llama ventana modal. La palabra "modal" significa que el visitante no puede interactuar con el resto de la página hasta que no haya cerrado la ventana. En este caso, hasta que se presione "Aceptar".

Un par de cuestiones, la primera es que no se ha especificado ningún caso especial para que aparezca la ventana. En el HTML simplemente se ha incorporado el fichero JavaScript y el navegador tan solo ha ejecutado todo el código encontrado en el fichero. Más adelante, cuando estudiemos los **eventos**, tendremos mayor control de cuándo será el momento exacto para que aparezca esta ventanita.

Por otro lado, antes de pulsar “Aceptar” observad cómo la página web se queda pausada. En el Chrome, se representan con un círculo de carga en la pestaña junto al nombre de la página web.

La información que podemos mostrar en un alert() es bastante limitada, texto y poco más. Más adelante veremos cómo dar mucho más contenido a mensajes emergentes.

Ya hemos visto cómo mostrar información al usuario, aparte de la web, en una ventana modal para llamar su atención. Ahora veremos cómo el usuario pueda dar información a la página o aplicación web. Esto lo podemos conseguir con prompt() que acepta dos parámetros como podemos ver en el siguiente ejemplo. El primer

www.grupostudium.com
informacion@grupostudium.com
954 539 952

valor es obligatorio y representa el mensaje que se mostrará en la ventana que aparece, de forma similar al mensaje de alert(). El segundo parámetro, opcional, indica el valor por defecto que aparecerá en el cuadro de texto del prompt() el cual podrá cambiar o no el usuario.

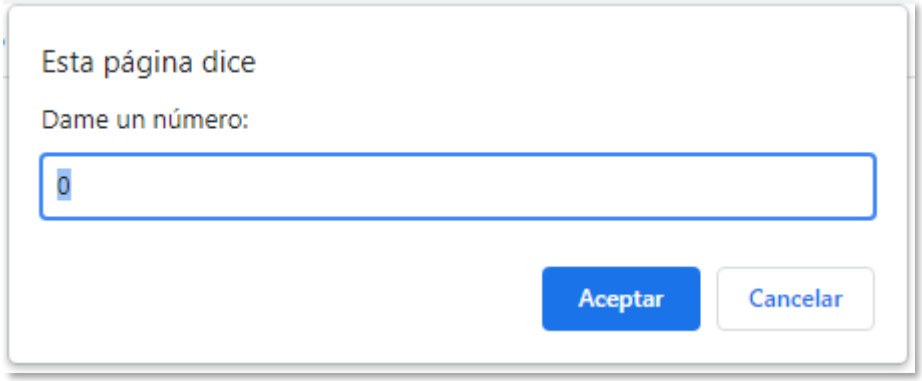
El ejemplo pide en una ventana modal (un prompt()) un número y luego, en otra ventana modal (un alert()) muestra el valor introducido.

El fichero index.html es el mismo que en el ejemplo anterior, pero el JavaScript difiere un poco:

```
let valor = prompt("Dame un número:", 0);
alert("Has metido el número " + valor);
```

index.js

En el navegador, en un primer momento tenemos:



Nos pide un número, y nos da uno por defecto, el 0. Si cambiamos el cero por un siete, al pulsar "Aceptar" ...

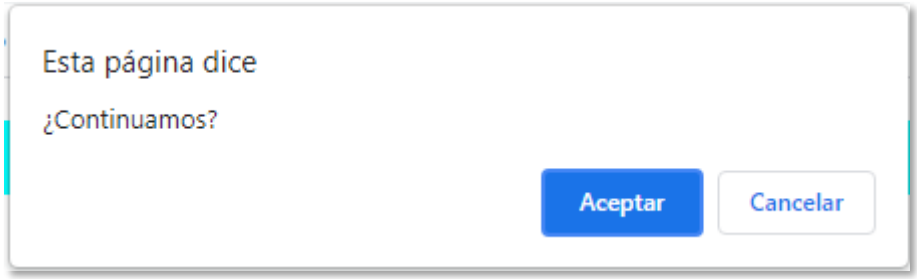
Por último, tenemos una ventana más sencilla si cabe que la anterior pues tiene la única finalidad de confirmar o no un mensaje dado.
Con el ejemplo lo vemos más claro.
El fichero index.html es el mismo, lo que cambia es el JavaScript.

```
let respuesta = confirm("¿Continuamos?");
alert(`Has contestado que ${respuesta}.`);
```

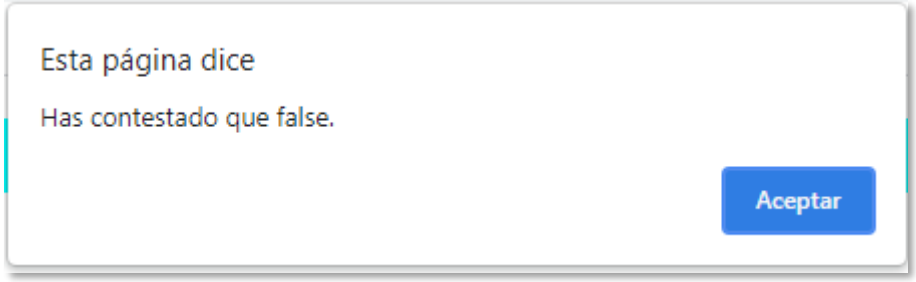
index.js

Probando el código en el navegador tenemos en primer lugar:

www.grupostudium.com
informacion@grupostudium.com
954 539 952



Y ahora, dependiendo de si pulsamos un botón u otro, se nos pueden indicar los siguientes mensajes, primero, al pulsar "Aceptar" y el segundo al pulsar "Cancelar":



Para mostrar el valor recogido de la ventana, se ha usado la variable "respuesta". Observad cómo se ha introducido el valor de dicha variable en el contexto del alert(): se han usado los acentos circunflejos ``` para englobar texto y variables, las cuales aparecen precedidas por el símbolo de dólar ($) y rodeadas de llaves { } para mostrar su contenido.

Hay dos limitaciones compartidas por todos los métodos anteriores:

- La ubicación exacta de la ventana modal está determinada por el navegador. Por lo general, está en el centro.
- El aspecto exacto de la ventana también depende del navegador. No podemos modificarlo.

Ese es el precio por la simplicidad. Aunque hay otras formas de mostrar ventanas más agradables y una interacción más rica con el visitante.

5.9 Eventos

Actualmente existen 19 de estos **eventos** predefinidos. Se pueden usar como disparadores para determinar cuándo un guion se debe ejecutar. Pueden ser

www.grupostudium.com
informacion@grupostudium.com
954 539 952

automáticos o que respondan a acciones del usuario: al entrar en la página, al pulsar un botón, al seleccionar una opción o al pasar el cursor del ratón sobre una imagen.

Pasamos a describirlos todos:

- **onblur**: Ocurre cuando el visitante abandona un elemento con el foco.
- **onchange**: Ocurre cuando el visitante ha cambiado el valor de un elemento de un formulario y ha dejado ese elemento, moviéndose fuera mediante tabulación o seleccionando otro elemento.
- **onclick**: Ocurre cuando el visitante hace clic en un elemento.
- **ondblclick**: Doble clic.
- **onfocus**: Ocurre cuando el usuario selecciona o tabula hacia un elemento.
- **onkeydown**: Ocurre incluso antes de que el visitante libera la tecla. Sirve para cualquier tecla como “Control”, “Borrar”, … Se usa en elementos de formulario en los que se pueda escribir.
- **onkeypress**: Ocurre cuando el usuario teclea cualquier carácter imprimible en un elemento de formulario. Se usa en elementos de formulario en los que se pueda escribir.
- **onkeyup**: Espera hasta que el usuario haya liberado la tecla. Se usa en elementos de formulario en los que se pueda escribir.
- **onload**: Ocurre cuando un navegador carga una página web. Se usa con <body>.
- **onmousedown**: Ocurre cuando el visitante apunta a un elemento HTML y pulsa el botón del ratón.
- **onmousemove**: Ocurre cuando el usuario mueve el puntero del ratón cuando ya está encima de un elemento.
- **onmouseout**: Ocurre cuando el visitante mueve el puntero fuera del elemento.
- **onmouseover**: Ocurre cuando el visitante apunta a un elemento o pasa por encima de él.
- **onmouseup**: Ocurre al liberarse el botón del ratón.
- **onreset**: Ocurre cuando el usuario hace clic en un botón de reseteo de un formulario, un reset.
- **onresize**: Se produce al redimensionar la pantalla, por cambiar la orientación del móvil, por ejemplo.
- **onselect**: Ocurre cuando el visitante selecciona algún fragmento de texto en un elemento de formulario.
- **onsubmit**: Ocurre cuando el visitante hace clic en un botón de envío de un formulario, un submit.
- **onunload**: Se produce cuando se realiza una descarga de una página web. Se usa con <body>.

Para probar cómo funcionan todos estos eventos, veremos a continuación, algunos ejemplos. Para ello usaremos nuestro VSC para escribir el código y para probar los eventos, en nuestro navegador abriremos las "Herramientas de Desarrollo", más concretamente nos fijaremos en la Consola, donde mostraremos mensajes con la instrucción de JavaScript console.log("Mensaje") como veremos a continuación.

Tendremos por una parte nuestro fichero index.html al que enlazaremos un fichero JavaScript con la funcionalidad necesaria llamado eventos.js dentro de la carpeta js.

En el listado anterior se han colocado los eventos por orden alfabético para su mejor presentación, pero en los ejemplos iremos de los más sencillos de entender a los más complejos, por necesitar más elementos para ver los resultados.

El primer ejemplo mostrará cómo funciona el evento onload que consiste, ni más ni menos, en producirse cuando se haya cargado la página web. Se asocia a la etiqueta <body>.

Comenzamos viendo el fichero eventos.js que contiene una función llamada evento que recibe un parámetro llamado tipo. Esta función simplemente mostrará por la consola de las "Herramientas de Desarrollo" un mensaje compuesto por una etiqueta fija, siempre la misma, y otra que viene del fichero HTML. De esta forma podremos usar la misma función para diferentes eventos.

El código sería:

```
function evento(tipo)
{
    console.log("Se ha producido el evento "+tipo);
}
```

eventos.js

Y ahora el fichero HTML:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>JavaScript</title>
</head>
<body onload="evento('onload')">
    <h1>JavaScript</h1>
    <script language="javascript" type="text/javascript" src=".\js\eventos.js">
    </script>
</body>
</html>
```

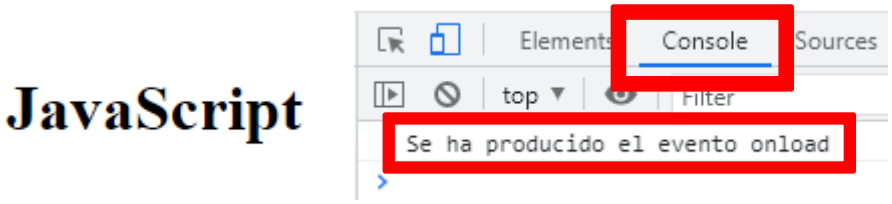
index.html

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Varias cosas a tener en cuenta:

1. La etiqueta <script> se puede colocar tanto en la cabecera <head> como en cualquier otro sitio de <body>. Se recomienda encarecidamente que se coloque al final del <body>, justo antes de la etiqueta </body>. El porqué de esta cuestión se verá más adelante cuando se estudie el concepto de **DOM**.
2. En la propia etiqueta <body> se ha añadido un atributo que precisamente es el propio **evento** con el que queremos trabajar, en este caso, onload al que le asignamos con el signo de igualdad lo que hay que hacer cuando el cuerpo de la página web se cargue. En este caso se ejecuta una función llamada **evento()** a la que enviamos como parámetro la etiqueta 'onload'.

Al guardar y mostrar la página en el navegador, con las "Herramientas de Desarrollo" habilitadas, obtenemos lo siguiente:



Igual que se produce un evento al cargar la página (onload), se produce otro evento cuando se descarga o pasa a segundo plano (onunload).

En el siguiente ejemplo, trabajaremos con el evento onclick sobre la cabecera <h1>, es decir, que cuando pulsemos en la palabra "JavaScript", se producirá el evento y se mostrará el mensaje de la consola.

A menos que se diga lo contrario, seguimos con el mismo fichero eventos.js.

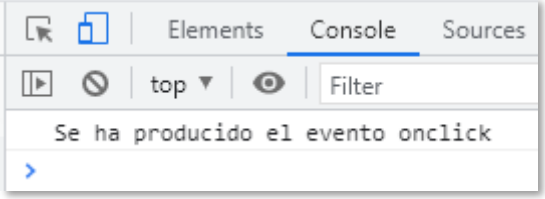
Fichero HTML:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>JavaScript</title>
</head>
<body>
    <h1 onclick="evento('onclick')">JavaScript</h1>
    <script language="javascript" type="text/javascript" src=".\js\eventos.js">
    </script>
</body>
</html>
```

index.html

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Ahora aplicamos el evento onclick a la etiqueta <h1> de manera que, en el navegador, al pulsar la cabecera se mostrará el mensaje correspondiente en la consola:



Pero cuidado, pues debemos tener en cuenta que la etiqueta <h1> es un elemento de bloque, es decir, que ocupa todo el ancho disponible, así que no solamente se producirá el evento sobre las letras que forman "JavaScript", sino en toda la línea hasta la parte derecha. Para probarlo, daremos estilo a la cabecera <h1> en un fichero CSS.

Fichero CSS:

```
h1
{
    background-color: aqua;
}
```

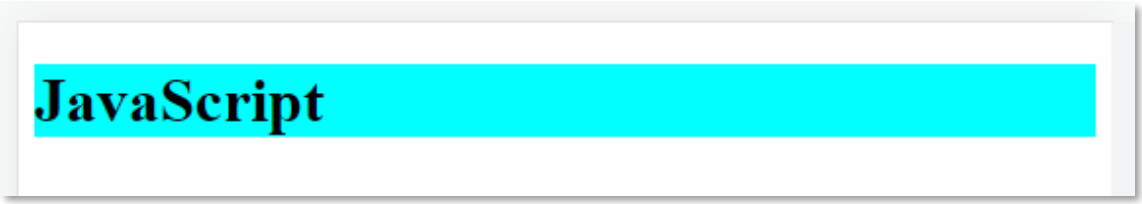
estilos.css

Y ahora lo enlazamos en el fichero HTML:

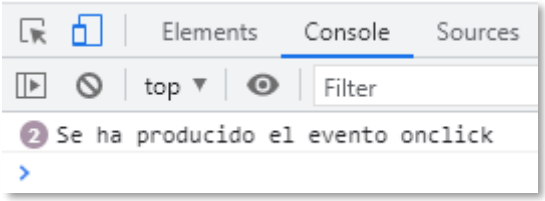
```
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="./css/estilos.css">
    <title>JavaScript</title>
</head>
```

index.html (fragmento de la cabecera)

En el navegador vemos cómo el elemento <h1> ocupa toda la fila, y en toda ella se responde al evento onclick:



www.grupostudium.com
informacion@grupostudium.com
954 539 952



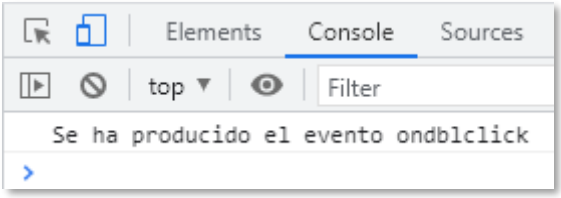
Estas pruebas se están haciendo en Google Chrome y cuando se repiten mensajes idénticos en la consola, no aparecen todos, sino que se muestra un número indicando las veces que aparece dicho mensaje.

Igualmente podemos probar con hacer doble click:

```
<h1 ondblclick="evento('ondblclick')">JavaScript</h1>
```

index.html (fragmento)

En el navegador:



NOTA: Cuidado al probar este ejemplo porque si tenemos desplegadas las herramientas de desarrollo (F12) la parte de la web se asume vista en un móvil y en vez de click o doble click se considerará hacer tap con el dedo y no saltará este evento. Para que funcione al 100%, ejecutar sin las herramientas de desarrollo, hacer doble click en el <h1> y luego desplegar la Consola para ver el resultado que vemos arriba.

Puede ocurrir lo mismo con los siguientes ejemplos, que incluimos de modo sucinto. ¡Probadlos TODOS!

Pasar el ratón por encima:

```
<h1 onmouseover="evento('onmouseover')">JavaScript</h1>
```

Pulsar el botón del ratón sobre el elemento <h1>:

```
<h1 onmousedown="evento('onmousedown')">JavaScript</h1>
```

Igualmente, para cuando se suelta el botón del ratón:

```
<h1 onmouseup="evento('onmouseup')">JavaScript</h1>
```

Para cuando se “saca” el cursor del ratón del elemento:

```
<h1 onmouseout="evento('onmouseout')">JavaScript</h1>
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

Por último, un ejemplo más elaborado para ver cómo funciona onmousemove:

```
<body onmousemove='evento(event)'>
    <h1>JavaScript</h1>
    <script language="javascript" type="text/javascript" src=".\js\index.js">
    </script>
</body>
```

index.html (fragmento)

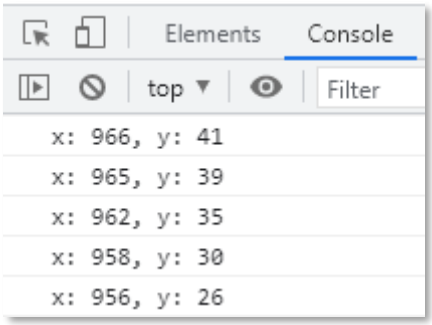
Ahora el evento lo asignamos al <body> para que se registre el movimiento del ratón en toda la web(Realmente, en lo que ocupan los elementos de la web que en ese caso particular es solamente el espacio ocupado por el único elemento presente, el <h1>).

Y el código:

```
function evento(event)
{
    let x = event.clientX;
    let y = event.clientY;
    console.log("x: "+x+", y: "+y);
}
```

index.js

Como podemos observar, en el HTML, al método evento() le mandamos un objeto llamado "event" que no es más que el evento que se ha producido, en nuestro caso, el movimiento del ratón. Este objeto es recibido en el método evento() y se le aplican dos funciones que nos dan, respectivamente, la posición X y la Y del cursor del ratón. Almacenamos esos valores en sendas variables y las mostramos por la consola:



Para los siguientes ejemplos vamos a usar el mismo fichero HTML, pero cambiando los eventos, ahora para trabajar con los de teclado.

```
<body onkeydown='evento(event)'>
```

index.html (fragmento)

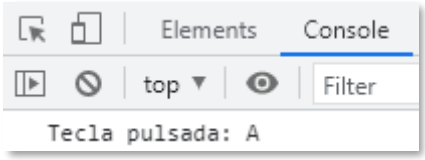
```
function evento(event)
{
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

```
    console.log("Tecla pulsada: "+String.fromCharCode(event.keyCode));
}
```

index.js

En este caso igualmente se envía el objeto "event" al método evento() donde obtenemos con la función correspondiente el valor de la tecla pulsada causante del evento (objeto evento):



Igualmente podemos proceder con onkeypress y onkeyup. El evento onkeydown reconoce todas las teclas, onkeypress todas salvo ALT, CTRL, SHIFT y ESC. Por último, onkeyup se produce cuando se suelta la tecla pulsada.

Para los siguientes ejemplos debemos tener una página un poco más elaborada compuesta por un formulario, pero lo haremos bien sencillo.

```
<body>
    <h1>JavaScript</h1>
    <form>
        Texto:<input type="text" placeholder="Aquí el texto..."
        onfocus="dentro();"
        onblur="fuera();"/>
    </form>
    <script language="javascript" type="text/javascript" src=".\js\index.js">
    </script>
</body>
```

index.html (fragmento)

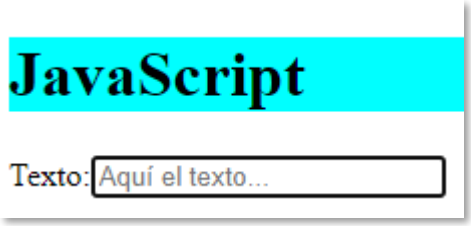
En este caso tenemos un simple cuadro de texto donde escribir al que hemos asociado los eventos de obtener el foco y perderlo, llamando respectivamente a una función JavaScript diferente para que actúe en consecuencia, a saber, mostrar un mensaje por consola:

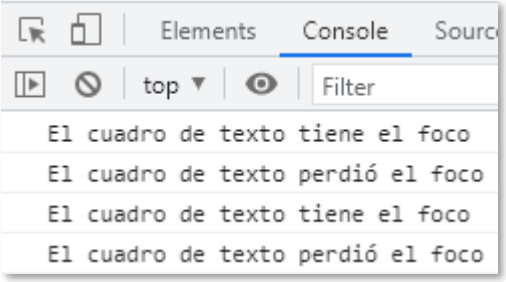
```
function dentro()
{
    console.log("El cuadro de texto tiene el foco");
}

function fuera()
{
    console.log("El cuadro de texto perdió el foco");
}
```

index.js

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Al probarlo en el navegador poniendo el cursor en el cuadro de texto y fuera, iremos generando los eventos que son capturados y que muestran los respectivos mensajes en la consola:





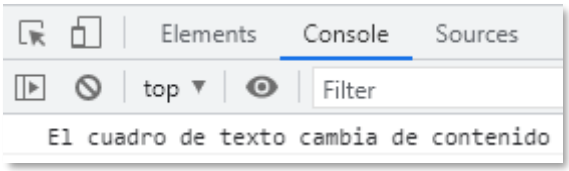
El siguiente ejemplo juego con el evento onchange que se produce cuando un cuadro de texto o similar, al obtener o perder el foco se detecta que ha cambiado el contenido.

```
<body>
    <h1>JavaScript</h1>
    <form>
        Texto:<input type="text" placeholder="Aquí el texto..."
        onchange="cambio();"/>
    </form>
    <script language="javascript" type="text/javascript" src=".\js\index.js">
    </script>
</body>
```

index.html (fragmento)

```
function cambio()
{
    console.log("El cuadro de texto cambia de contenido");
}
```

index.js



Tenemos ahora el evento que se produce al seleccionar algún elemento de opciones como los checkboxes, los radio button o los desplegables o choices.

Probaremos con un cuadro de texto desplegable:

```
<body>
    <h1>JavaScript</h1>
    <form>
        Provincia:
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

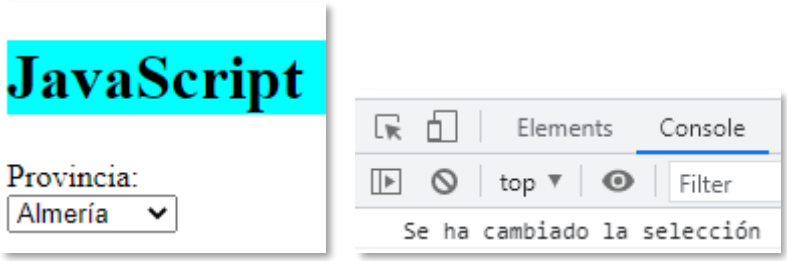
```
        <br/>
        <select name="provincia" onchange="cambio()">
            <option value="0">Elegir una</option>
            <option value="1">Almería</option>
            <option value="2">Cádiz</option>
            <option value="3">Córdoba</option>
            <option value="4">Granada</option>
            <option value="5">Huelva</option>
            <option value="6">Jaen</option>
            <option value="7">Málaga</option>
            <option value="8">Sevilla</option>
        </select>
    </form>
    <script language="javascript" type="text/javascript" src=".\js\index.js">
    </script>
</body>
```

index.html (fragmento)

Y ahora el JavaScript:

```
function cambio()
{
    console.log("Se ha cambiado la selección");
}
```

index.js



Los siguientes eventos a estudiar son onreset y onsubmit. El primero está asociado al <input> de tipo "reset", es decir, un botón que reinicia todos los elementos de un formulario. Tiene poca utilidad.

Por el contrario, el segundo está asociado al botón de tipo "submit", sin duda, el elemento más usado en los formularios pues lanza la validación implícita, la propia de cada tipo de <input> y otros controles similares, así como realiza el envío de toda la información suministrada por el usuario al servidor que tratará dichos datos para hacer una Consulta, realizar un Alta, Modificar un Registro o Borrarlo.

Además, en cuanto a JavaScript se refiere, también está muy vinculado a la ejecución de programas, aplicaciones o scripts.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Veamos un ejemplo sencillo en el que saludamos con el nombre introducido en un cuadro de texto al pulsar un botón "Aceptar":

```
<body>
    <h1>JavaScript</h1>
    <form onsubmit="mostrarNombre();">
        Tu nombre:<input type="text" id="miNombre"/>
        <br/>
        <input type="submit" value="Aceptar" />
        <input type="reset" value="Limpiar"/>
    </form>
    <script language="javascript" type="text/javascript" src=".\js\index.js">
    </script>
</body>
```

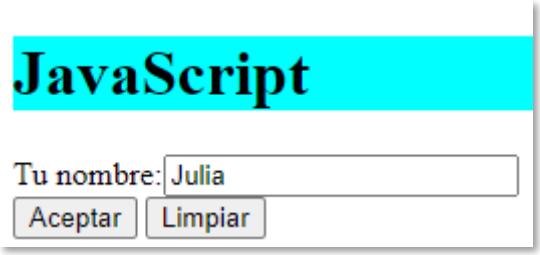
index.html (fragmento)

El JavaScript sería:

```
function mostrarNombre()
{
    let nombre = document.getElementById("miNombre");
    alert("Hola " + nombre.value);
}
```

index.js

En el navegador...



Al pulsar "Aceptar":

Comentar que la instrucción ...

```
let nombre = document.getElementById("miNombre");
```

Crea la variable llamada "nombre" cuyo contenido será el elemento identificado con el id="miNombre" (Ver HTML). Se refiere al elemento completo, a todo lo que supone

www.grupostudium.com
informacion@grupostudium.com
954 539 952

el objeto <input>. Sin embargo, nosotros, lo que queremos es el valor de lo que el usuario haya escrito; por eso, en el mensaje alert() accedemos a esa información mediante el atributo "value":

```
alert("Hola " + nombre.value);
```

Pensad que el elemento <input> tiene muchos atributos. Solamente necesitamos, en esta ocasión, el valor, el "value".

Por último, onresize se produce al redimensionar el elemento sobre el que se aplica. Si lo hacemos sobre el <body> se contempla toda la web, como en el siguiente ejemplo:

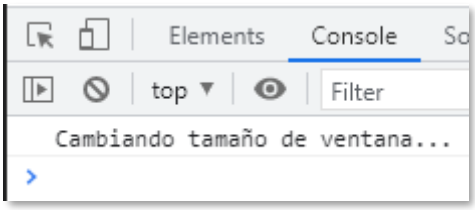
```
<body onresize="avisar();">
    <h1>JavaScript</h1>
    <script language="javascript" type="text/javascript" src=".\js\index.
js">
    </script>
</body>
```

index.html (fragmento)

```
function avisar()
{
    console.log("Cambiando tamaño de ventana...");
}
```

index.js

En la consola del navegador cuando vayamos cambiando el tamaño de la ventana:



www.grupostudium.com
informacion@grupostudium.com
954 539 952

5.10 Modelo de Objetos del Documento

Una **página web** es un documento HTML que es interpretado por los navegadores en forma gráfica, pero también permiten el acceso al código.

El **Modelo de Objetos del Documento** (DOM) permite ver el mismo documento de otra manera, describiendo el contenido del documento como un conjunto de objetos que un programa JavaScript puede actuar sobre ellos.

La creación del **Document Object Model** es una de las innovaciones que más ha influido en el desarrollo de las páginas web dinámicas y de las aplicaciones web más complejas.

DOM permite a los programadores web **acceder** y **manipular** las páginas XHTML como si fueran documentos XML. De hecho, DOM se diseñó originalmente para manipular de forma sencilla los documentos XML.

A pesar de sus orígenes, DOM se ha convertido en una utilidad disponible para la mayoría de lenguajes de programación (Java, PHP, JavaScript) y cuyas únicas diferencias se encuentran en la forma de implementarlo.

Acorde al W3C el Modelo de Objetos del Documento es una **interfaz de programación de aplicaciones** (API) para documentos validos HTML y documentos XML bien construidos. Define la estructura lógica de los documentos y el modo en que se accede y manipula.

El DOM permite un acceso a la estructura de una página HTML mediante el mapeo de los elementos de esta página en un **árbol de nodos**. Cada elemento se convierte en un nodo y cada porción de texto en un nodo de texto.

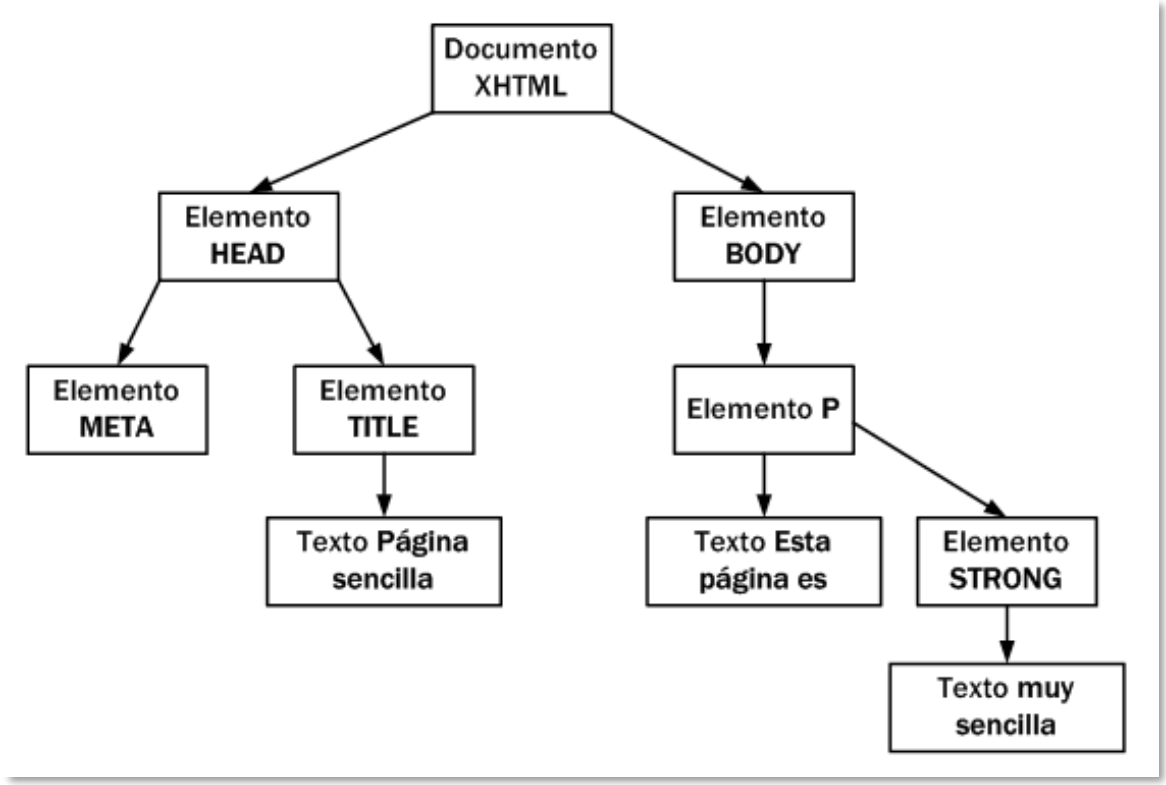
DOM transforma todos los documentos XHTML en un conjunto de elementos llamados **nodos**, que están interconectados y que representan los contenidos de las páginas web y las relaciones entre ellos.

Veamos un ejemplo:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Página sencilla</title>
</head>
<body>
    <p>Esta página es <strong>muy sencilla</strong></p>
</body>
</html>
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

Se transforma en el siguiente árbol de nodos:



En el esquema anterior, cada rectángulo representa un nodo DOM y las flechas indican las relaciones entre nodos. Dentro de cada nodo, se ha incluido su tipo (que se verá más adelante) y su contenido.

La **raíz del árbol** de nodos de cualquier página XHTML siempre es la misma: un nodo de tipo especial denominado "Documento".

A partir de ese nodo raíz, cada etiqueta XHTML se transforma en un nodo de tipo "Elemento". La conversión de etiquetas en nodos se realiza de forma jerárquica. De esta forma, del nodo raíz solamente pueden derivar los nodos HEAD y BODY. A partir de esta derivación inicial, cada etiqueta XHTML se transforma en un nodo que deriva del nodo correspondiente a su "etiqueta padre".

La transformación de las etiquetas XHTML habituales genera dos nodos: el primero es el nodo de tipo "Elemento" (correspondiente a la propia etiqueta XHTML) y el segundo es un nodo de tipo "Texto" que contiene el texto encerrado por esa etiqueta XHTML.

Así, la siguiente etiqueta XHTML:

```
<title>Página sencilla</title>
```

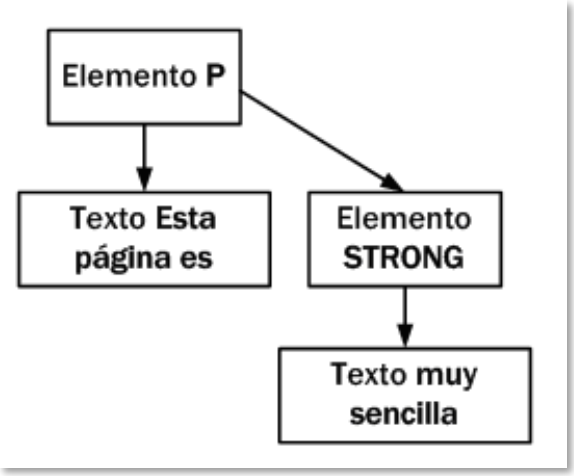
Genera los siguientes dos nodos:

www.grupostudium.com
informacion@grupostudium.com
954 539 952

De la misma forma, la siguiente etiqueta XHTML:

```
<p>Esta página es <strong>muy sencilla</strong></p>
```

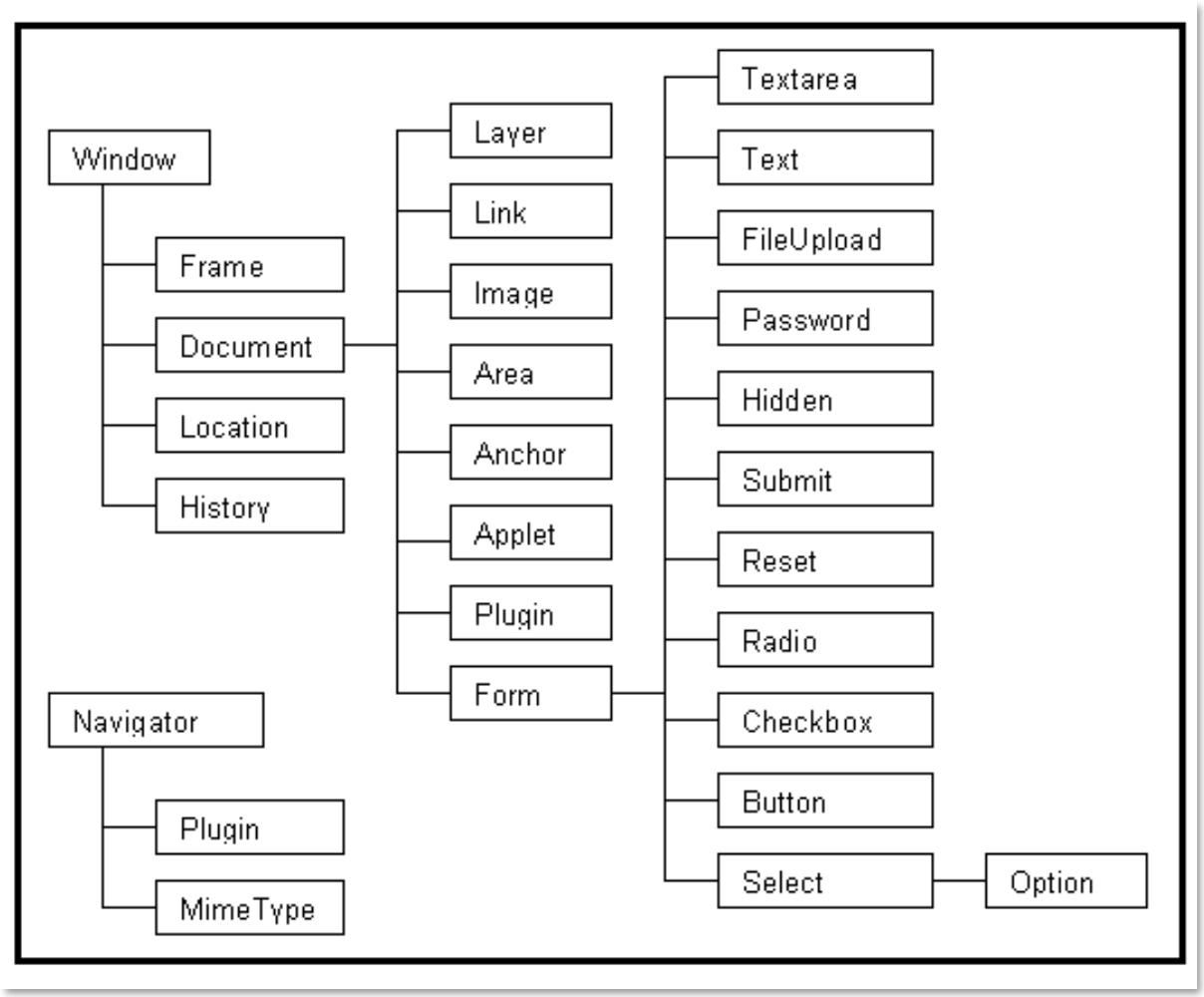
Genera:



Como se puede suponer, las páginas XHTML habituales producen árboles con miles de nodos. Aun así, el proceso de transformación es **rápido y automático**, siendo las funciones proporcionadas por DOM (que se verán más adelante) las únicas que permiten acceder a cualquier nodo de la página de forma sencilla e inmediata.

En la siguiente imagen podemos ver el modelo de objetos común de los navegadores:

www.grupostudium.com
informacion@grupostudium.com
954 539 952



Tipos de nodos

La especificación completa de DOM define 12 tipos de nodos, aunque las páginas XHTML habituales se pueden manipular manejando solamente cuatro o cinco tipos de nodos:

- **Document**, nodo raíz del que derivan todos los demás nodos del árbol.
- **Element**, representa cada una de las etiquetas XHTML. Se trata del único nodo que puede contener atributos y el único del que pueden derivar otros nodos.
- **Attr**, se define un nodo de este tipo para representar cada uno de los atributos de las etiquetas XHTML, es decir, uno por cada par atributo=valor.
- **Text**, nodo que contiene el texto encerrado por una etiqueta XHTML.
- **Comment**, representa los comentarios incluidos en la página XHTML.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Los otros tipos de nodos existentes que no se van a considerar son DocumentType, CDataSection, DocumentFragment, Entity, EntityReference, ProcessingInstruction y Notation.

Acceso a los nodos

Una vez construido automáticamente el árbol completo de nodos DOM, ya es posible utilizar las funciones DOM para **acceder** de forma directa a cualquier nodo del árbol. Como acceder a un nodo del árbol es equivalente a acceder a "un trozo" de la página, una vez construido el árbol, ya es posible manipular de forma sencilla la página: acceder al valor de un elemento, establecer el valor de un elemento, mover un elemento de la página, crear y añadir nuevos elementos, etc.

Las funciones que proporciona DOM para acceder a un nodo a través de sus nodos padre consisten en acceder al nodo raíz de la página y después a sus nodos hijos y a los nodos hijos de esos hijos y así sucesivamente hasta el último nodo de la rama terminada por el nodo buscado. Sin embargo, cuando se quiere acceder a un nodo específico, es mucho más rápido acceder directamente a ese nodo y no llegar hasta él descendiendo a través de todos sus nodos padre.

Por ese motivo, no se van a presentar las funciones necesarias para el acceso jerárquico de nodos y se muestran solamente las que permiten acceder de forma directa a los nodos.

Por último, es importante recordar que el acceso a los nodos, su modificación y su eliminación solamente es posible cuando el árbol DOM ha sido construido completamente, es decir, después de que la página XHTML se cargue por completo. Más adelante se verá cómo asegurar que un código JavaScript solamente se ejecute cuando el navegador ha cargado entera la página XHTML.

getElementsByTagName()

Como sucede con todas las funciones que proporciona DOM, la función *getElementsByTagName()* tiene un nombre muy largo, pero que lo hace autoexplicativo.

La función *getElementsByTagName(nombreEtiqueta)* obtiene todos los elementos de la página XHTML cuya etiqueta sea igual que el parámetro que se le pasa a la función.

El siguiente ejemplo muestra cómo obtener todos los párrafos de una página XHTML:

```
let parrafos = document.getElementsByTagName("p");
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

El valor que se indica delante del nombre de la función (en este caso, *document*) es el nodo a partir del cual se realiza la búsqueda de los elementos. En este caso, como se quieren obtener todos los párrafos de la página, se utiliza el valor *document* como punto de partida de la búsqueda.

El valor que devuelve la función es un array con todos los nodos que cumplen la condición de que su etiqueta coincide con el parámetro proporcionado. El valor devuelto es un array de nodos DOM, no un array de cadenas de texto o un array de objetos normales. Por lo tanto, se debe procesar cada valor del array de la forma que se muestra en las siguientes secciones.

De este modo, se puede obtener el primer párrafo de la página de la siguiente manera:

```
let primerParrafo = parrafos[0];
```

De la misma forma, se podrían recorrer todos los párrafos de la página con el siguiente código:

```
for(let i=0; i<parrafos.length; i++)
{
    let parrafo = parrafos[i];
}
```

La función *getElementsByTagName()* se puede aplicar de forma recursiva sobre cada uno de los nodos devueltos por la función. En el siguiente ejemplo, se obtienen todos los enlaces del primer párrafo de la página:

```
let parrafos = document.getElementsByTagName("p");
let primerParrafo = parrafos[0];
let enlaces = primerParrafo.getElementsByTagName("a");
```

getElementsByName()

La función *getElementsByName()* es similar a la anterior, pero en este caso se buscan los elementos cuyo atributo *name* sea igual al parámetro proporcionado. En el siguiente ejemplo, se obtiene directamente el único párrafo con el nombre indicado:

```
let parrafoEspecial = document.getElementsByName("especial");
```

```
<p name="prueba">...</p>
<p name="especial">...</p>
<p>...</p>
```

Normalmente el atributo *name* es único para los elementos HTML que lo definen, por lo que es un método muy práctico para acceder directamente al nodo deseado. En el caso de los elementos HTML *radiobutton*, el atributo *name* es común a todos los *radiobutton* que están relacionados, por lo que la función devuelve una colección de elementos.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

getElementById()

La función *getElementById()* es la más utilizada cuando se desarrollan aplicaciones web dinámicas. Se trata de la función preferida para acceder directamente a un nodo y poder leer o modificar sus propiedades.

La función *getElementById()* devuelve el elemento XHTML cuyo atributo "id" coincide con el parámetro indicado en la función. Como el atributo "id" debe ser único para cada elemento de una misma página, la función devuelve únicamente el nodo deseado:

```
let cabecera = document.getElementById("cabecera");
```

```
<div id="cabecera">
    <a href="/" id="logo">...</a>
</div>
```

La función *getElementById()* es tan importante y tan utilizada en todas las aplicaciones web, que casi todos los ejemplos y ejercicios que siguen la utilizan constantemente.

Creación y eliminación de nodos

Acceder a los nodos y a sus propiedades (que se verá más adelante) es sólo una parte de las manipulaciones habituales en las páginas. Las otras operaciones habituales son las de crear y eliminar nodos del árbol DOM, es decir, crear y eliminar "trozos" de la página web.

Creación de elementos XHTML simples

Como se ha visto, un elemento XHTML sencillo, como por ejemplo un párrafo, genera dos nodos: el primer nodo es de tipo *Element* y representa la etiqueta <p> y el segundo nodo es de tipo *Text* y representa el contenido textual de la etiqueta <p>.

Por este motivo, crear y añadir a la página un nuevo elemento XHTML sencillo consta de cuatro pasos diferentes:

- Creación de un nodo de tipo *Element* que represente al elemento.
- Creación de un nodo de tipo *Text* que represente el contenido del elemento.
- Añadir el nodo *Text* como nodo hijo del nodo *Element*.
- Añadir el nodo *Element* a la página, en forma de nodo hijo del nodo correspondiente al lugar en el que se quiere insertar el elemento.

De este modo, si se quiere añadir un párrafo simple al final de una página XHTML, es necesario incluir el siguiente código JavaScript:

```
function crear()
```

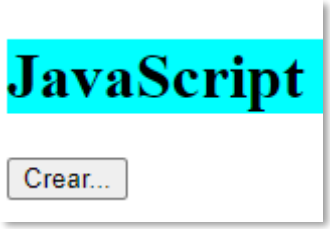

www.grupostudium.com
informacion@grupostudium.com
954 539 952

```
{
    // Crear nodo de tipo Element
    let parrafo = document.createElement("p");
    // Crear nodo de tipo Text
    let contenido = document.createTextNode("Hola Mundo!");
    // Añadir el nodo Text como hijo del nodo Element
    parrafo.appendChild(contenido);
    // Añadir el nodo Element como hijo de la pagina
    document.body.appendChild(parrafo);
}
```

Y el fichero HTML (parte):

```
<body>
    <h1>JavaScript</h1>
    <button onclick="crear();">Crear...</button>
    <script language="javascript" type="text/javascript" src=".\js\index.js">
    </script>
</body>
```

En el navegador partimos de:



Y cada vez que pulsemos el botón, se genera un nodo o elemento nuevo (hijo) y se añade a lo que ya hay:

www.grupostudium.com
informacion@grupostudium.com
954 539 952

El proceso de creación de nuevos nodos puede llegar a ser tedioso, ya que implica la utilización de tres funciones DOM:

- **createElement(etiqueta)**: crea un nodo de tipo *Element* que representa al elemento XHTML cuya etiqueta se pasa como parámetro.
- **createTextNode(contenido)**: crea un nodo de tipo *Text* que almacena el contenido textual de los elementos XHTML.
- **nodoPadre.appendChild(nodoHijo)**: añade un nodo como hijo de otro nodo. Se debe utilizar al menos dos veces con los nodos habituales: en primer lugar, se añade el nodo *Text* como hijo del nodo *Element* y a continuación se añade el nodo *Element* como hijo de algún nodo de la página.

Eliminación de nodos

Afortunadamente, eliminar un nodo del árbol DOM de la página es mucho más sencillo que añadirlo. En este caso, solamente es necesario utilizar la función *removeChild()*. Completemos:

```
let parrafo = document.getElementById("provisional");
parrafo.parentNode.removeChild(parrafo);
```

```
<p id="provisional">...</p>
```

La función *removeChild()* requiere como parámetro el nodo que se va a eliminar. Además, esta función debe ser invocada desde el elemento padre de ese nodo que se quiere eliminar. La forma más segura y rápida de acceder al nodo padre de un elemento es mediante la propiedad nodoHijo.parentNode.

Así, para eliminar un nodo de una página XHTML se invoca a la función *removeChild()* desde el valor *parentNode* del nodo que se quiere eliminar. Cuando se elimina un nodo, también se eliminan automáticamente todos los nodos hijos que tenga, por lo que no es necesario borrar manualmente cada nodo hijo.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Acceso directo a los atributos XHTML

Una vez que se ha accedido a un nodo, el siguiente paso natural consiste en acceder y/o modificar sus atributos y propiedades. Mediante DOM, es posible acceder de forma sencilla a todos los atributos XHTML y todas las propiedades CSS de cualquier elemento de la página.

Los atributos XHTML de los elementos de la página se transforman automáticamente en propiedades de los nodos. Para acceder a su valor, simplemente se indica el nombre del atributo XHTML detrás del nombre del nodo.

El siguiente ejemplo obtiene de forma directa la dirección a la que enlaza el enlace:

```
let enlace = document.getElementById("enlace");
alert(enlace.href); // muestra http://www...com
```

```
<a id="enlace" href="http://www...com">Enlace</a>
```

En el ejemplo anterior, se obtiene el nodo DOM que representa el enlace mediante la función *document.getElementById()*. A continuación, se obtiene el atributo *href* del enlace mediante enlace.href. Para obtener por ejemplo el atributo "id", se utilizaría enlace.id.

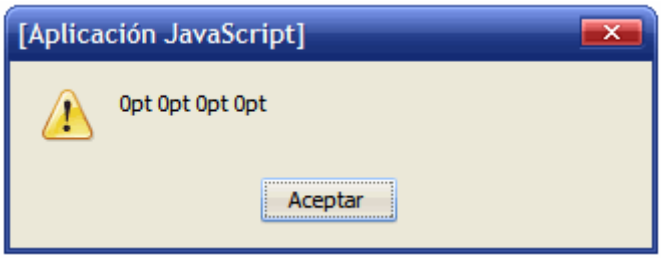
Las propiedades CSS no son tan fáciles de obtener como los atributos XHTML. Para obtener el valor de cualquier propiedad CSS del nodo, se debe utilizar el atributo **style**. El siguiente ejemplo obtiene el valor de la propiedad *margin* de la imagen:

```
let imagen = document.getElementById("imagen");
alert(imagen.style.margin);
```

```
<img id="imagen" style="margin:0; border:0;" src="logo.png" />
```

Aunque el funcionamiento es homogéneo entre distintos navegadores, los resultados no son exactamente iguales, como muestran las siguientes imágenes que son el resultado de ejecutar el código anterior en distintos navegadores:

www.grupostudium.com
informacion@grupostudium.com
954 539 952



Si el nombre de una propiedad CSS es **compuesto**, se accede a su valor modificando ligeramente su nombre:

```
let parrafo = document.getElementById("parrafo");
alert(parrafo.style.fontWeight);  // Muestra "bold"
```

```
<p id="parrafo" style="font-weight: bold;">...</p>
```

La transformación del nombre de las propiedades CSS compuestas consiste en **eliminar** todos los guiones medios (-) y escribir en mayúscula la letra siguiente a cada guion medio. A continuación, se muestran algunos ejemplos:

- ***font-weight*** se transforma en *fontWeight*
- ***line-height*** se transforma en *lineHeight*
- ***border-top-style*** se transforma en *borderTopStyle*
- ***list-style-image*** se transforma en *listStyleImage*

El único atributo XHTML que no tiene el mismo nombre en XHTML y en las propiedades DOM es el atributo **class**. Como la palabra *class* está reservada por JavaScript, no es posible utilizarla para acceder al atributo *class* del elemento XHTML. En su lugar, DOM utiliza el nombre **className** para acceder al atributo *class* de XHTML:

```
let parrafo = document.getElementById("parrafo");
alert(parrafo.class); // Muestra "undefined"
alert(parrafo.className); // Muestra "normal"
```

```
<p id="parrafo" class="normal">...</p>
```

Para finalizar este apartado, vamos a realizar el siguiente ejemplo en el que tendremos un formulario con dos cuadros de texto donde pondremos números y un botón que calculará la suma, mostrando el resultado en otro cuadro de texto.

Fichero HTML:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

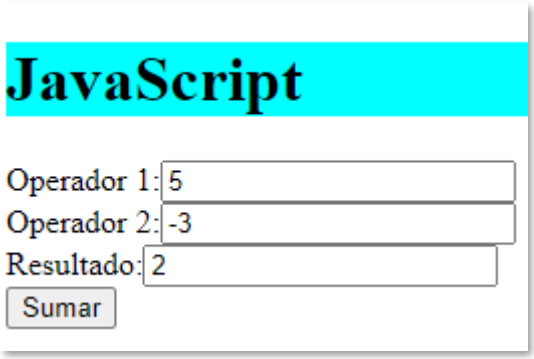
```
    <link rel="stylesheet" href="./css/estilos.css">
    <title>JavaScript</title>
</head>
<body>
    <h1>JavaScript</h1>
    Operador 1:<input type="number" id="operador1" />
    <br />
    Operador 2:<input type="number" id="operador2" />
    <br />
    Resultado:<input type="number" id="resultado" />
    <br />
    <button onclick="sumar();">Sumar</button>
    <script language="javascript" type="text/javascript" src=".\js\index.js">
    </script>
</body>
</html>
```

Fichero js:

```
function sumar()
{
    var operador1 = document.getElementById('operador1').value;
    var operador2 = document.getElementById('operador2').value;
    document.getElementById("resultado").value = parseInt(operador1) + parseInt(operador2);
}
```

La función parseInt() transforma en entero la cadena pasada como parámetro.

En el navegador:



Objeto "window"

Este objeto representa cualquier ventana abierta por el navegador. Actualmente esas ventanas se asimilan a "pestañas" del navegador.

Por ejemplo, el siguiente botón, abrirá una nueva pestaña con la web indicada:

```
<button onclick="window.open('http://www.w3c.es');">Pulsar</button>
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

Es el objeto predeterminado del DOM, de manera que cuando colocamos una instrucción como:

```
alert("Hola, mundo!");
```

Realmente estamos llamando a:

```
window.alert("Hola, mundo!");
```

De esta forma, algunas de las funciones vistas con anterioridad no son tales funciones, sino que se trata en realidad de métodos del objeto *window*.

Otros métodos interesantes de *window* son confirm y prompt, como pudimos ver en el apartado 8.8.

Objeto "location"

Es hijo del objeto *window*. Contiene toda la información referente a la localización u origen del documento XHTML que se muestra en una determinada ventana o pestaña.

Su propiedad más importante es href, y es tanto de lectura como de escritura de manera que podemos mostrar su valor:

```
alert(location.href);
```

Pero también podemos hacer que cambie:

```
location.href="http://www.w3c.es";
```

Objeto "history"

Cuando se navega por la red usando un navegador, se van guardando las páginas que se van visitando en una memoria llamada **historial**. El propio navegador tiene herramientas para avanzar o retroceder en dicho historial o incluso muestra con detalle las visitas realizadas en el día de hoy, ayer, la semana pasada, etc.

Con JavaScript podemos también "movernos" por dicho historial gracias al objeto **history**.

Este objeto tiene una única propiedad llamada length que nos muestra el tamaño de dicho historial, o lo que es lo mismo, cuántas páginas se han visitado. Y también dispone de tres métodos: back(), forward() y go(valor). El primero permite ir hacia atrás en el historial, el segundo permite ir hacia delante y go permite, mediante la inclusión de un número entero como parámetro, ir hacia delante o atrás, tantas posiciones en el historial como indica el número. Así, *history.go(1)* equivaldría a forward(), *history.go(-1)* equivaldría a back(). Cualquier otro valor supondría el salto en dicha cantidad en las posiciones del historial.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Objeto "screen"

Este objeto nos puede informar sobre características de la pantalla del usuario.

Veamos con un ejemplo las principales propiedades:

```
function mostrar()
{
    console.log(screen.width); // Ancho de la pantalla
    console.log(screen.height); // Alto de la pantalla
    console.log(screen.colorDepth); // Profundidad de colores
}
```

Objeto "document"

Este objeto representa al propio documento que se está mostrando en la ventana del navegador. Se usa para acceder a sus propiedades, así como a los elementos que lo constituyen. Siempre está disponible para cualquier ventana. Sus propiedades y métodos son muy sencillos:

- **title** contiene el título de documento.
- **bgColor** contiene el color de fondo para el documento.
- **fgColor** indica el color para el texto del documento.
- **linkColor** indica el color con el que se resaltarán los hiperenlaces de la página.
- **aLinkColor** especifica el color con que se resaltarán los hiperenlaces activos de la página.
- **vLinkColor** especifica el color con que se resaltarán los hiperenlaces de la página ya visitados y estén en la caché.
- **lastModified** devuelve la fecha de la última modificación del documento actual.

El objeto document posee unas propiedades que son a su vez matrices o colecciones de objetos. Dichas propiedades terminan en -s. Estas colecciones nos permiten acceder a los elementos de la página del tipo que indica su nombre. Veamos algunos ejemplos.

Mostrar todas las anclas o enlaces internos:

```
let anclas = "";
for(i=0; i<document.anchors.length;i++)
{
    anclas += (i+1) + ".- " + document.anchors[i].name + "\n";
}
alert(anclas);
```

Mostrar todos los enlaces:

www.grupostudium.com
informacion@grupostudium.com
954 539 952

```
let enlaces = "";
for(i=0; i<document.links.length;i++)
{
    enlaces += (i+1) + ".- " + document.links[i].href + "\n";
}
alert(enlaces);
```

Mostrar todas las imágenes:

```
let imagenes = "";
for(i=0; i<document.images.length;i++)
{
    imagenes += (i+1) + ".- " + document.images[i].src + "\n";
}
alert(imagenes);
```

Un ejemplo práctico

Para acabar, veremos un ejemplo más elaborado donde pondremos a trabajar algunos de los conceptos que hemos estudiado.

Antes de ver el ejemplo debemos explicar el uso de una función que ejecuta otra función JavaScript al transcurrir cierto tiempo expresado en milisegundos:

```
setTimeout("Función", Tiempo);
```

El código trabajará con una frase que aparecerá letra a letra. Primero el fichero HTML (fragmento):

```
<body onload ="animar();">
    <h1>JavaScript</h1>
    <form>
        <input type="text" name="TextoDinamico" value="" size="120" />
    </form>
    <script language="javascript" type="text/javascript" src=".\js\index.js">
    </script>
</body>
```

Y ahora el fichero JavaScript:

```
let velocidad = 200;
let letras;
let TextoActual = "";
let n = -1;
function animar()
{
    letras = new Array();
    let texto = "Hola que tal!";
    for (i = 0; i < texto.length; i++)
    {
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

```
        letras[i] = texto.charAt(i);
    }
    mueveLetras();
}
```

```
function mueveLetras()
{
    n++;
    TextoActual += letras[n];
    document.forms[0].TextoDinamico.value = TextoActual;
    if (n == letras.length - 1)
    {
        n = -1;
        TextoActual = "";
    }
    setTimeout("mueveLetras()", velocidad);
}
```

5.11 jQuery

Se trata de una librería de JavaScript que facilita mucho el desarrollo de aplicaciones web y sobre todo mejoran mucho el aspecto gráfico y de eventos, al incorporar multitud de elementos para un diseño más atractivo y rápido, así como todo un elenco de efectos gráficos y de eventos que hacen más dinámicas y agradables a nuestras aplicaciones web.

jQuery

Las definiciones de **jQuery** rondan siempre a que simplifica enormemente el desarrollo de aplicaciones JavaScript y además es fácil de aprender.

Para adentrarnos en el mundo jQuery solamente tenemos que conocer HTML, CSS y JavaScript.

Como su eslogan indica, jQuery es “escribir menos, hacer más”. La idea es recoger funcionalidad de muchas líneas de JavaScript y envolverlas en otra funcionalidad más simple y sencilla.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

También simplifica mucho el trabajo de JavaScript con Ajax o con el manejo del DOM.

También posee gran cantidad de plug-ins que aumentan aún más su potencial.

Existen otros frameworks de JavaScript, pero este se recomienda por ser el más popular y el más expandible. Grandes corporaciones usan jQuery para sus desarrollos como Google, Microsoft, IBM o Netflix, por indicar unos cuantos.

Comenzando

Lo primero que debemos saber es que el equipo de desarrollo de jQuery procura siempre que todo sea compatible al máximo posible con los navegadores principales.

jQuery se puede usar de dos modos: descargando los ficheros y acoplarlos a nuestros proyectos, o bien acoplarlos directamente desde un CDN.

Tanto en un caso como en el otro, tenemos siempre dos versiones: una para producción, compacta y reducida, y otra de desarrollo, más legible y expandida. La de producción se suele identificar con un subíndice .min que denota el mínimo espacio del fichero. Por ejemplo, jquery-3.6.0.min.js correspondería a la versión de producción 3.6.0.

Ambas versiones se pueden descargar desde la página principal de jQuery.

La librería jQuery es un fichero simple de JavaScript, que para incorporarlo a nuestro proyecto web, simplemente debemos usar la etiqueta HTML <script> tal como se muestra a continuación:

```
<scriptlanguage="javascript"type="text/javascript"src=".\js\jquery-3.6.0.js"></script>
```

Si no queremos descargar el fichero, usaremos los CDN disponibles:

```
<scriptsrc="https://code.jquery.com/jquery-3.6.0.js"integrity="sha256-
H+K7U5CnXl1h5ywQfKtSj8PCmoN9aaq30gDh27Xc0jk="crossorigin="anonymous"></sc
ript>
```

Una gran ventaja de usar el jQuery alojado de Google o Microsoft es que muchos usuarios ya han descargado jQuery de Google o Microsoft cuando visitan otro sitio. Como resultado, se cargará desde la memoria caché cuando visiten su sitio, lo que lleva a un tiempo de carga más rápido. Además, la mayoría de los CDN se asegurarán de que una vez que un usuario solicite un archivo, se le envíe desde el servidor más cercano a ellos, lo que también permite un tiempo de carga más rápido.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Sintaxis

Con jQuery tú seleccionas (query) los elementos HTML y trabajas con ellos.

La sintaxis básica es...

```
$(selector).acción()
```

...donde el signo $ es el indicativo de jQuery, el selector identifica el elemento HTML sobre el que se está actuando y la acción es la tarea a realizar con dicho elemento o debido a dicho elemento.

Algunos ejemplos sencillos:

```
$(this).hide() // Oculta el elemento actual
$("p").hide() // Oculta todos los elementos <p>
$(".test").hide() // Oculta el element con el atributo class="test"
$("#test").hide() // Oculta el element con el atributo id="test"
```

¿A qué se parece? A los selectores de CSS.

Evento DocumentReady

Todos los códigos jQuery van dentro del evento documentready de la siguiente forma:

```
$(document).ready(function()
{
   // Código jQuery ...
});
```

Esto es para evitar que se ejecute cualquier código jQuery antes de que el documento termine de cargarse (Esté listo).

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Es una buena práctica esperar a que el documento esté completamente cargado y listo antes de trabajar con él.

Estos son algunos ejemplos de acciones que pueden fallar si se ejecutan los métodos antes de que el documento se cargue por completo:

- Tratando de ocultar un elemento que aún no está creado
- Intentando obtener el tamaño de una imagen que aún no está cargada

El equipo de jQuery también ha creado un método aún más corto para el evento documentready:

```
$(function()
{
   // Código jQuery ...
});
```

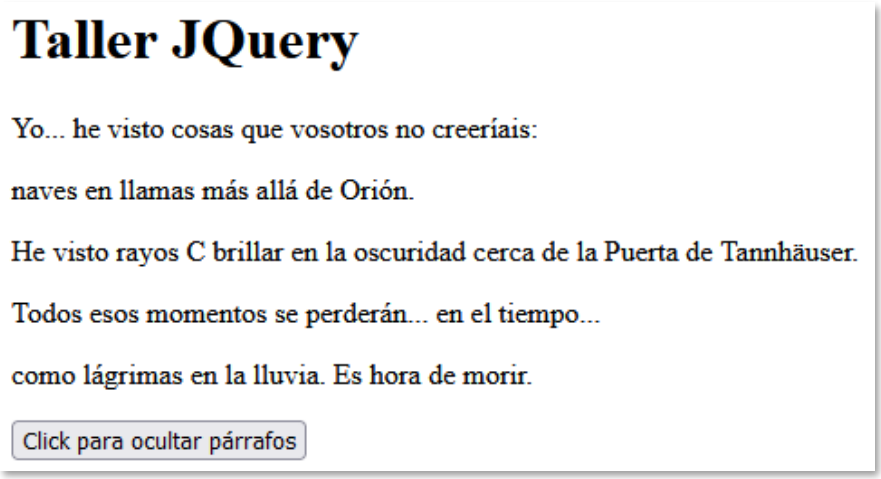
Selectores

Los **selectores** de jQuery son uno de los elementos más importantes de la librería. Permiten acceder a los elementos de una página web, así como su manipulación.

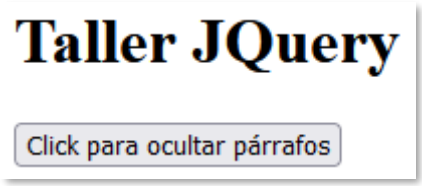
Dicho acceso se puede realizar por su name, por su id, por su clase, por el tipo, por sus atributos, por los valores de dichos atributos y muchos más. Se basa en los selectores de CSS, además de los suyos propios.

Todos los selectores de jQuery comienzan con el signo de dólar seguido del paréntesis. $().

Si queremos seleccionar por tipo de elemento, lo indicaremos con el tipo entre comillas. En el siguiente ejemplo se ocultarán todos los elementos <p> al pulsar un botón:



www.grupostudium.com
informacion@grupostudium.com
954 539 952




```
<head>
    <metacharset="UTF-8">
    <metahttp-equiv="X-UA-Compatible"content="IE=edge">
    <metaname="viewport"content="width=device-width, initial-scale=1.0">
    <title>JavaScript: JQuery</title>
</head>
<body>
    <h1>Taller JQuery</h1>
   <p>Yo... he visto cosas que vosotros no creeríais: </p>
   <p>naves en llamas más allá de Orión. </p>
   <p>He visto rayos C brillar en la oscuridad cerca de la Puerta de Tannhäuser.</p>
   <p>Todos esos momentos se perderán... en el tiempo... </p>
   <p>como lágrimas en la lluvia. Es hora de morir.</p>
   <button>Click para ocultar párrafos</button>
   <scriptlanguage="javascript"type="text/javascript"src=".\js\jquery-
3.6.0.js"></script>
   <scriptlanguage="javascript"type="text/javascript"src=".\js\index.js"></script>
</body>
</html>
```

index.html

```
$(function()
{
    $("button").on("click", function()
    {
        $("p").hide();
    });
});
```

index.js

Practicar: Añadir un segundo botón y la funcionalidad correspondiente para que, al pulsarlo, aparezcan los párrafos.

Para identificar un elemento por el id, debemos indicarlo con la almohadilla(#). En el siguiente ejemplo, solamente se ocultará el elemento con id="encabezado":

```
<body>
    <h1id="encabezado">Taller JQuery</h1>
    <p>Yo... he visto cosas que vosotros no creeríais: </p>
    <p>naves en llamas más allá de Orión. </p>
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

```
    <p>He visto rayos C brillar en la oscuridad cerca de la Puerta de Tannhäuser.</p>
    <p>Todos esos momentos se perderán... en el tiempo... </p>
    <p>como lágrimas en la lluvia. Es hora de morir.</p>
    <button>Click para ocultar "encabezado"</button>
    <scriptlanguage="javascript"type="text/javascript"src=".\js\jquery-
3.6.0.js"></script>
    <scriptlanguage="javascript"type="text/javascript"src=".\js\index.js"></script>
</body>
```

index.html (Fragmento)

```
$(function()
{
    $("button").on("click", function()
    {
        $("#encabezado").hide();
    });
});
```

index.js

Para seleccionar los elementos de una determinada clase, usaremos el operador punto (.). En el siguiente ejemplo, se ocultarán todos los elementos que tengan el atributo class="secundarios":

```
<body>
    <h1id="encabezado">Taller JQuery</h1>
    <p>Yo... he visto cosas que vosotros no creeríais: </p>
    <pclass="secundario">naves en llamas más allá de Orión. </p>
    <p>He visto rayos C brillar en la oscuridad cerca de la Puerta de Tannhäuser.</p>
    <pclass="secundario">Todos esos momentos se perderán... en el tiempo... </p>
    <p>como lágrimas en la lluvia. Es hora de morir.</p>
    <button>Click para ocultar "secundario"</button>
    <scriptlanguage="javascript"type="text/javascript"src=".\js\jquery-
3.6.0.js"></script>
    <scriptlanguage="javascript"type="text/javascript"src=".\js\index.js"></script>
</body>
```

index.html (Fragmento)

```
$(function()
{
    $("button").on("click", function()
    {
        $(".secundario").hide();
    });
});
```

index.js

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Algunos ejemplos más:

```
$("*")  // Seleccionar todos los elementos
$(this)  // Seleccionar el elemento actual
$("p.intro")    // Seleccionar todos los elementos <p> con class="intro”
$("p:first")    // Seleccionar el primer <p>
$("ulli:first")    // Seleccionar el primer <li> del primer <ul>
$("ulli:first-child")  // Seleccionar el primer <li> de cada <ul>
$("[href]") // Seleccionar todos los elementos con atributo href
$("a[target='_blank']") // Seleccionar todos los elementos <a> con un
atributo target igual a "_blank"
$("a[target!='_blank']") // Seleccionar todos los elementos <a> con un
atributo target diferente a "_blank"
$(":button")    // Seleccionar todos los <button> y los elementos <input>
del tipo  type="button"
$("tr:even")    // Seleccionar todos los elementos <tr> pares
$("tr:odd") // Los impares
```

Eventos

jQuery está diseñado para responder a los eventos que se produzcan en una página HTML.

Algunos eventos comunes:

Ratón	Teclado	Formularios	Pestaña/Página
click	keypress	submit	load
dblclick	keydown	change	resize
mouseenter	keyup	focus	scroll
mouseleave		blur	unload

Para asignar un evento de click a todos los párrafos de una página, podemos hacer los siguiente:

```
$("p").on("click", function(){});
```

Lo siguiente que debemos hacer es indicar qué debe ocurrir cuando se produzca dicho evento. En jQuery debemos pasar una función:

```
$("p").on("click", function()
{
  // Acciones a realizar
});
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

Veamos algunos ejemplos:

Código	Interpretación

`$("p").on("click", function()`
`{`
`  $(this).hide();`
`});`
	Oculta el párrafo sobre el que se ha pulsado.

`$("p").on("dblclick", function()`
`{`
`  $(this).hide();`
`});`
	Oculta el párrafo sobre el que se ha pulsado dos veces.

`$("#p1").on("mouseenter", function()`
`{`
`  alert("¡Entraste en p1!");`
`});`
	Muestra un mensaje al entrar en el elemento con id=p1.

`$("#p1").on("mouseleave", function()`
`{`
`  alert("¡Adiós! ¡Dejaste p1!");`
`});`
	Muestra un mensaje al abandonar el elemento con id=p1.

`$("#p1").on("mousedown", function()`
`{`
`  alert("¡Pulsaste sobre p1!");`
`});`
	Muestra un mensaje al pulsar cualquier botón del ratón en el elemento con id=p1.

`$("#p1").on("mouseup", function()`
`{`
`  alert("¡Soltaste sobre p1!");`
`});`
	Muestra un mensaje al soltar cualquier botón del ratón en el elemento con id=p1.

`$("#p1").hover(function()`
`{`
`  alert("¡Entraste en p1!");`
`},function()`
`{`
`  alert("¡Adiós! ¡Dejaste p1!");`
`});`
	La primera función se ejecuta al entrar en el elemento con id=p1 y la segunda función, al abandonarlo.

`$("input").focus(function()`
`{`
`  $(this).css("background-color",`
`"#00ffff");`
`});`
	La función se ejecuta cuando el campo del formulario obtiene el foco.

`$("input").blur(function()`
`{`
`  $(this).css("background-color",`
`"#ffffff");`
`});`
	La función se ejecuta cuando el campo del formulario pierde el foco.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

`$("p").on({ mouseenter:function() { $(this).css("background-color", "lightgray"); }, mouseleave:function() { $(this).css("background-color", "lightblue"); }, click:function() { $(this).css("background-color", "yellow"); } });`	Varios eventos asociados a la vez.

Efectos

Veremos ahora una serie de efectos que incorpora jQuery para personalizar la presentación de los elementos de nuestras páginas web.

Mostrar y ocultar elementos

Se usan las funciones hide() y show():

```
<h1id="encabezado">Taller JQuery</h1>
<p>Este párrafo aparece y desaparece con los botones.</p>
<buttonid="hide">Ocultar</button>
<buttonid="show">Mostrar</button>
```

index.html (Fragmento)

```
$("#hide").on("click", function(){
    $("p").hide();
});

$("#show").on("click", function(){
    $("p").show();
});
```

index.js

Las funciones hide() y show() se pueden usar con unos parámetros determinados como son velocidad y/o callback, donde velocidad indica la velocidad de aparecer o desaparecer. Puede ser “fast”, “slow” o una cantidad de milisegundos.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

El segundo parámetro indica una función a realizar tras la operación de mostrar u ocultar.

```
$("#hide").on("click", function(){
    $("p").hide("fast", function() {
        alert( "Animación completada." );})
});

$("#show").on("click", function(){
    $("p").show("slow", function() {
        alert( "Animación completada." );})
});
```

index.js

Existe también la función toggle() que cambia el estado de visibilidad de un elemento ocultando si está visible o haciendo visible si está oculto.

```
$("#toggle").on("click", function(){
    $("p").toggle("slow", function() {
        alert( "Animación completada." );})
});
```

index.js

Igualmente tiene velocidad y/o callback.

Aparecer y desaparecer

El siguiente efecto consiste en ocultar o mostrar un elemento HTML pero paulatinamente, modificando la transparencia de dicho elemento. Estamos hablando de las funciones fadeIn(velocidad, callback) y fadeOut(velocidad, callback) y también para cambiar de un estado a otro fadeToggle(velocidad, callback).

```
  <pclass="1">Este párrafo aparece y desaparece con los botones.</p>
    <pclass="2">Este párrafo aparece y desaparece con los botones de otra
forma.</p>
    <pclass="3">Este párrafo aparece y desaparece con los botones de otra
forma más.</p>
    <buttonid="fadeIn">Aparecer</button>
    <buttonid="fadeOut">Desaparecer</button>
    <buttonid="toggle">Intercambiar</button>
```

index.html (Fragmento)

```
$("#fadeIn").on("click", function(){
    $(".1").fadeIn();
    $(".2").fadeIn("slow");
    $(".3").fadeIn(3000);
});
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

```
$("#fadeOut").on("click", function(){
    $(".1").fadeOut();
    $(".2").fadeOut("slow");
    $(".3").fadeOut(3000);
});

$("#toggle").on("click", function(){
    $(".1").fadeToggle();
    $(".2").fadeToggle("slow");
    $(".3").fadeToggle(3000);
});
```

index.js

Por último, existe la posibilidad de establecer cuánto se quiere hacer desaparecer jugando con el grado de opacidad/transparencia:

```
$("#almost").on("click", function(){
    $(".1").fadeTo("slow", 0.15);
    $(".2").fadeTo("slow", 0.4);
    $(".3").fadeTo("slow", 0.7);
});
```

index.js

Los valores de la opacidad/transparencia se especifica entre 0 y 1. Admite igualmente los otros parámetros de velocidad y callback.

Deslizamiento

Este efecto hace aparecer y desaparecer elementos, pero simulando una cortina.

Se puede deslizar hacia abajo (aparecer) o hacia arriba (desaparecer). Pero también alternar uno con el otro. Podemos trabajar con los parámetros de velocidad y de callback.

```
<img class="1"src="images/jquery.jpg"/>
<br/>
<button id="slideDown">Aparecer</button>
<button id="slideUp">Desaparecer</button>
```

index.html (Fragmento)

```
$("#slideDown").on("click", function(){
    $(".1").slideDown();
});

$("#slideUp").on("click", function(){
    $(".1").slideUp();
});
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

index.js

Animaciones

Los mejores efectos en jQuery los podemos obtener con las animaciones personalizadas tal como veremos en este apartado. El método animate() admite los siguientes parámetros:

```
animate({parámetrosvarios}, velocidad, callback);
```

Los "parámetros varios" definen las propiedades CSS a animar.

El siguiente ejemplo mueve un a la derecha hasta que la propiedad izquierda tiene el valor de 250px:

```
<div style="background:#98bf21;height:100px;width:100px;position:absolute;">
</div>
<br/><br/><br/><br/><br/><br/>
<button id="move">Mover</button>
```

index.html (Fragmento)

```
$("#move").on("click", function(){
    $("div").animate({right:'250px'});
});
```

index.js

Por defecto, todos los elementos HTML tienen una posición estática y no se pueden mover.

Para manipular la posición, recuerde establecer primero la propiedad de posición CSS del elemento como relativa, fija o absoluta.

Para manipular diferentes propiedades:

```
$("#move").on("click", function(){
    $("div").animate({right:'250px',
    opacity:'0.5',
    height:'150px',
    width:'150px'
    });
});
```

index.js

Todas las propiedades CSS se pueden animar de esta forma, pero debemos tener en cuenta una regla: todos los nombres de propiedad deben ser camel-cased

www.grupostudium.com
informacion@grupostudium.com
954 539 952

cuando se usan con el método animate(): Se tendrá que escribir paddingLeft en lugar de padding-left, marginRight en lugar de margin-right, y así sucesivamente.

Además, la animación de color no está incluida en la biblioteca básica jQuery.

También es posible definir valores relativos (el valor es entonces relativo al valor actual del elemento). Esto se hace colocando + = o - = delante del valor:

```
$("#move").on("click", function(){
    $("div").animate({left:'+=150px',
    height:'+=150px',
    width:'+=150px'
    });
});
```

index.js

Incluso puede especificar el valor de animación de una propiedad como "show", "hide" o "toggle":

```
$("#move").on("click", function(){
    $("div").animate({left:'+=150px',
    height:'toggle',
    width:'toggle'
    });
});
```

index.js

Por defecto, jQuery viene con funcionalidad de cola para animaciones. Esto significa que si escribe múltiples llamadas a animate() una después de la otra, jQuery crea una cola "interna" con estas llamadas a métodos. Luego ejecuta las llamadas animadas una a una.

Por lo tanto, si desea realizar diferentes animaciones una después de la otra, aprovechamos la funcionalidad de la cola:

```
$("#move").on("click", function(){
    vardiv = $("div");
    div.animate({height:'300px', opacity:'0.4'}, "slow");
    div.animate({width:'300px', opacity:'0.8'}, "slow");
    div.animate({height:'100px', opacity:'0.4'}, "slow");
    div.animate({width:'100px', opacity:'0.8'}, "slow");
});
```

index.js

Otro ejemplo de colas:

```
<divstyle="background:#98bf21;height:100px;width:100px;position:absolute;">Texto
</div>
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

```
<br/><br/><br/><br/><br/><br/>
<buttonid="move">Mover</button>
```

index.html (Fragmento)

```
$("#move").on("click", function(){
    vardiv = $("div");
    div.animate({left:'100px'}, "slow");
    div.animate({fontSize:'3em'}, "slow");
});
```

index.js

Parar

El método stop() se usa para parar una animación o efecto antes de que acabe.

Admite dos parámetros, **stopAll** y **goToEnd**. El primero indica si la cola debe ser anulada también o no. Por defecto es false, es decir, que solamente se para una animación o efecto de la cola. El segundo parámetro indica si debemos o no completar la animación actual inmediatamente. Por defecto es false también.

```
<divstyle="background:#98bf21;height:100px;width:100px;position:absolute;">Texto
</div>
<br/><br/><br/><br/><br/><br/>
<buttonid="move">Mover</button>
<buttonid="stop">Parar</button>
```

index.html (Fragmento)

```
$("#move").on("click", function(){
    vardiv = $("div");
    div.animate({left:'1000px'}, "slow");
    div.animate({fontSize:'3em'}, "slow");
});
$("#stop").on("click", function(){
    vardiv = $("div");
    div.stop();
});
```

index.js

Funciones de callback

Una función de **callback** se ejecuta después de que el efecto actual esté terminado por completo

Las instrucciones de JavaScript se ejecutan línea por línea. Sin embargo, con los efectos, la siguiente línea de código se puede ejecutar, aunque el efecto no haya finalizado. Esto puede crear errores. Para evitar esto, puede crear una función de callback. Una función de callback se ejecuta después de que el efecto actual finaliza.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

El siguiente ejemplo tiene un parámetro de callback que es una función que se ejecutará después de que se complete el efecto de animación:

```
<divstyle="background:#98bf21;height:100px;width:100px;position:absolute;">
</div>
<br/><br/><br/><br/><br/><br/>
<buttonid="move">Mover</button>
```

index.html (Fragmento)

```
$("#move").on("click", function(){
    vardiv = $("div");
    div.animate({left:'1000px'}, "slow", function()
    {
        alert("Animación finalizada");
    });
});
```

index.js

Y ahora observad la diferencia con respecto a:

```
$("#move").on("click", function(){
    vardiv = $("div");
    div.animate({left:'1000px'}, "slow");
    alert("Animación finalizada");
});
```

index.js

El ejemplo anterior no tiene parámetro de devolución de llamada, y se mostrará el cuadro de alerta antes de que se complete el efecto de animación.

Encadenamiento

En jQuery se pueden **encadenar** acciones y métodos. Nos permite ejecutar múltiples acciones sobre el mismo elemento dentro de una única instrucción.

Esta forma de proceder agiliza la ejecución de nuestras instrucciones por parte de los navegadores que solamente tienen que buscar el elemento una vez.

El formato es sencillo: añadir una acción a la anterior usando el operador punto (.):

```
$("#p1").css("color", "red").slideUp(2000).slideDown(2000);
```

Al encadenar, la línea de código podría ser bastante larga. Sin embargo, jQuery no es muy estricto con la sintaxis; puede formatearlo como desee, incluidos los saltos de línea y las sangrías:

```
$("#p1")
.css("color", "red")
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

```
.slideUp(2000)
.slideDown(2000);
```

Interacción con HTML

jQuery contiene poderosos métodos para manipular y cambiar elementos HTML y sus respectivos atributos. También estamos hablando del DOM.

Comencemos con los tres principales métodos para interactuar:

- text(), para obtener o establecer el contenido de texto del elemento seleccionado.
- html(), para obtener o establecer el contenido del elemento seleccionado.
- val(), para obtener o establecer el valor de un campo de un formulario.

Obtener información

El siguiente ejemplo muestra cómo obtener contenido con los métodos jQuery text() y html():

```
<pid="test">Volverán las oscuras golondrinas...</p>
<buttonid="boton1">Mensaje de test()</button>
<buttonid="boton2">Mensaje de html()</button>
```

index.html (Fragmento)

```
$("#boton1").on("click", function(){
    alert("Texto: " + $("#test").text());
});
$("#boton2").on("click", function(){
    alert("Texto: " + $("#test").html());
});
```

index.js

El siguiente ejemplo muestra cómo obtener el valor de un campo de entrada de un formulario con el método jQuery val():

```
<form>
    <inputtype="text"id="test"/>
</form>
<br/>
<buttonid="boton">Mensaje de val()</button>
```

index.html (Fragmento)

```
$("#boton").on("click", function(){
    alert("Texto: " + $("#test").val());
});
```

index.js

www.grupostudium.com
informacion@grupostudium.com
954 539 952

El método attr() se usa para obtener valores de atributo.

El siguiente ejemplo muestra cómo obtener el valor del atributo *href* en un enlace identificado por 'w3s':

```
<ahref="https://www.w3schools.com/"id="w3s">Ir a w3schools</a>
<br/><br/>
<buttonid="boton">Mensaje de attr()</button>
```

index.html (Fragmento)

```
$("#boton").on("click", function(){
    alert("Texto: " + $("#w3s").attr('href'));
});
```

index.js

Establecer información

Antes hemos estudiado cómo obtener información de los elementos del DOM. Ahora veremos cómo cambiar o establecer dicha información, datos, o valores.

Igualmente usaremos los mismos tres métodos para establecer o modificar el contenido de un elemento:

- text(), para obtener o establecer el contenido de texto del elemento seleccionado.
- html(), para obtener o establecer el contenido del elemento seleccionado.
- val(), para obtener o establecer el valor de un campo de un formulario.

El siguiente ejemplo demuestra su uso:

```
<divid="test1"></div>
<divid="test2"></div>
<inputtype="text"id="test3"/>
<br/><br/>
<buttonid="boton1">Establecer text</button>
<buttonid="boton2">Establecer html</button>
<buttonid="boton3">Establecer val</button>
```

index.html (Fragmento)

```
$("#boton1").on("click", function(){
    $("#test1").text("¡Hola mundo!");
});
$("#boton2").on("click", function(){
    $("#test2").html("<b>¡Hola mundo!</b>");
});
$("#boton3").on("click", function(){
    $("#test3").val("Dolly Duck");
});
```

index.js

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Los tres métodos jQuery anteriores también vienen con una función de callback. La función de callback tiene dos parámetros: el índice del elemento actual en la lista de elementos seleccionados y el valor original (anterior). A continuación, devuelve la cadena que desea utilizar como el nuevo valor de la función.

El siguiente ejemplo muestra text() y html() con una función de callback:

```
<divid="test1"></div>
<divid="test2"></div>
<br/><br/>
<buttonid="boton1">Establecer text</button>
<buttonid="boton2">Establecer html</button>
```

index.html (Fragmento)

```
$("#boton1").on("click", function(){
    $("#test1").text(function(i, origText)
    {
        return"Viejo Texto: " + origText + " Nuevo Texto (index: " + i + ")";
    });
});

$("#boton2").on("click", function(){
    $("#test2").html(function(i, origText)
    {
       return"Viejo HTML: " + origText + " Nuevo HTML: ¡Hola <b>mundo!</b>(index: "
+ i + ")";
   });
});
```

index.js

El método attr() también se usa para establecer o cambiar valores de atributos de elementos HTML.

Ejemplo:

```
<ahref=""id="w3s">Ir a w3schools</a>
<br/><br/>
<buttonid="boton">Establecer attr()</button>
```

index.html (Fragmento)

```
$("#boton").on("click", function(){
    $("#w3s").attr("href", "https://www.w3schools.com/jquery/");
});
```

index.js

www.grupostudium.com
informacion@grupostudium.com
954 539 952

El método attr() también le permite establecer múltiples atributos al mismo tiempo.

```
$("#boton").on("click", function(){
    $("#w3s").attr({
        "href" :"https://www.w3schools.com/jquery/",
        "title" :"Tutorial de W3Schools jQuery"
    });
});
```

index.js

El método attr(), también viene con una función de callback.

Añadiendo nuevo contenido HTML

Con jQuery es fácil añadir nuevos elementos y contenido a nuestras páginas web HTML.

Tenemos cuatro métodos para lograr esto: append(), prepend(), after() y before().

Veámoslos detenidamente.

El método append() inserta contenido al *final* del elemento HTML seleccionado:

```
<p>Texto</p>
<br/><br/>
<buttonid="boton">Añadir</button>
```

index.html (Fragmento)

```
$("#boton").on("click", function(){
    $("p").append("Más texto añadido al p.");
});
```

index.js

Sin embargo, el método *prepend()* inserta al *principio*:

```
$("#boton").on("click", function(){
    $("p").prepend("Más texto añadido al p.");
});
```

index.js

En los dos ejemplos anteriores, solo hemos insertado texto HTML al principio o al final de los elementos HTML seleccionados.

Sin embargo, tanto el método append() como prepend() pueden tomar un número infinito de nuevos elementos como parámetros. Los nuevos elementos se pueden generar con texto HTML (como hemos hecho en los ejemplos anteriores), con jQuery o con código JavaScript y elementos DOM.

En el siguiente ejemplo, creamos varios elementos nuevos. Los elementos se crean con texto HTML, jQuery, JavaScript y DOM. Luego agregamos los nuevos elementos

www.grupostudium.com
informacion@grupostudium.com
954 539 952

al texto con el método append() (esto habría funcionado igualmente para prepend()):

```
$("#boton").on("click", function(){
    vartxt1 = "<p>Texto</p>";                // Crear element con HTML
    vartxt2 = $("<p></p>").text("Texto");    // Crear con jQuery
    vartxt3 = document.createElement("p");   // Crear con DOM
    txt3.innerHTML = "Texto";
    $("body").append(txt1, txt2, txt3);       // Añadir todos los elementos nuevos
});
```

index.js

El método after() inserta contenido DESPUÉS de los elementos HTML seleccionados.

El método before() inserta contenido ANTES de los elementos HTML seleccionados.

Se pueden agregar con estas funciones igual que vimos con append() y prepend().

Eliminar elementos

Igual que añadimos, podemos eliminar elementos. Disponemos de los métodos remove() y empty(). El primero elimina el elemento seleccionado y todos sus hijos, el segundo solamente borra los hijos del elemento seleccionado, pero no éste.

Ejemplos:

```
$("#div1").remove();
$("#div1").empty();
```

El método remove() también acepta un parámetro, que le permite filtrar los elementos que se eliminarán.

El parámetro puede ser cualquiera de las sintaxis del selector jQuery.

El siguiente ejemplo elimina todos los elementos <p> con class = "test":

```
$("p").remove(".test");
```

Y el siguiente borra todos los elementos <p> con class="test" o class="demo":

```
$("p").remove(".test, .demo");
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

Manipulación de elementos CSS

Con jQuery, manipular elementos CSS es fácil (como casi todo). Tiene varios métodos para la manipulación de CSS. Veremos los siguientes métodos:

- addClass() - Agrega una o más clases a los elementos seleccionados
- removeClass() - Elimina una o más clases de los elementos seleccionados
- toggleClass() - Alterna entre agregar / eliminar clases de los elementos seleccionados
- css() - Establece o devuelve el atributo de estilo indicado

Para los siguientes ejemplos, usaremos esta hoja de estilo:

```
.important
{
    font-weight: bold;
    font-size: xx-large;
}

.blue
{
    color: blue;
}
```

estilos.css

El siguiente ejemplo muestra cómo agregar atributos de clase a diferentes elementos. Por supuesto, puede seleccionar múltiples elementos al agregar clases:

```
<!DOCTYPEhtml>
<htmllang="es">

<head>
    <metacharset="UTF-8">
    <metahttp-equiv="X-UA-Compatible"content="IE=edge">
    <metaname="viewport"content="width=device-width, initial-scale=1.0">
    <linkrel="stylesheet"type="text/css"href="/css/estilos.css"media="screen"/>
    <title>JavaScript: JQuery</title>
</head>

<body>
    <h1>Taller JQuery</h1>
    <h2>Añadiendo estilos con JQuery</h2>
    <p>Texto</p>
    <br/><br/>
    <div>Sección</div>
    <br/><br/>
    <buttonid="boton">Añadir estilo</button>

    <scriptlanguage="javascript"type="text/javascript"src=".\js\jquery-
3.6.0.js"></script>
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

```
    <scriptlanguage="javascript"type="text/javascript"src=".\js\index.js"></script>
</body>

</html>
```

index.html

```
$("#boton").on("click", function(){
    $("h1, h2, p").addClass("blue");
    $("div").addClass("important");
});
```

index.js

También puede especificar múltiples clases dentro del método addClass():

```
$("div").addClass("important blue");
```

El siguiente ejemplo muestra cómo eliminar un atributo de clase específico de diferentes elementos:

```
<h1>Taller JQuery</h1>
<h2>Añadiendo estilos con JQuery</h2>
<p>Texto</p>
<br/><br/>
<div>Sección</div>
<br/><br/>
<buttonid="boton1">Añadir estilo</button>
<buttonid="boton2">Quitar estilo</button>
```

index.html (Fragmento)

```
$("#boton1").on("click", function(){
    $("h1, h2, p").addClass("blue");
    $("div").addClass("important blue");
});
$("#boton2").on("click", function(){
    $("h1, h2, p").removeClass("blue");
    $("div").removeClass("important blue");
});
```

index.js

El siguiente ejemplo mostrará cómo usar el método toggleClass(). Este método alterna entre agregar y eliminar clases de los elementos seleccionados:

```
<h1>Taller JQuery</h1>
<h2>Añadiendo estilos con JQuery</h2>
<p>Texto</p>
<br/><br/>
<div>Sección</div>
<br/><br/>
<buttonid="boton">Alternar estilo</button>
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

index.html (Fragmento)

```
$("#boton").on("click", function(){
    $("h1, h2, p").toggleClass("blue");
    $("div").toggleClass("important blue");
});
```

index.js

Método css()

El método css() establece o devuelve una o más propiedades de estilo para los elementos seleccionados.

Para obtener el valor de una propiedad específica CSS, usaremos:

```
css("nombrePropiedad");
```

El siguiente ejemplo devolverá el valor de color de fondo del PRIMER elemento coincidente:

```
$("p").css("background-color");
```

Para establecer una propiedad de CSS determinada, use la siguiente sintaxis:

```
css("nombrePropiedad","valor");
```

El siguiente ejemplo establecerá el valor de color de fondo para TODOS los elementos coincidentes:

```
$("p").css("background-color", "yellow");
```

Para establecer múltiples propiedades de CSS, use la siguiente sintaxis:

```
css ({"nombrePropiedad ":" valor ", " nombrePropiedad ":" valor ", ...});
```

El siguiente ejemplo establecerá un color de fondo y un tamaño de fuente para TODOS los elementos coincidentes:

```
$("p").css({"background-color":"yellow", "font-size":"200%"});
```

Dimensiones

Disponemos de varios métodos para trabajar con las dimensiones:

- width(), establece u obtiene el ancho del elemento
- height(), establece u obtiene el alto del elemento

www.grupostudium.com
informacion@grupostudium.com
954 539 952

- innerWidth(), establece u obtiene al ancho de la ventana, sin incluir barras de herramientas.
- innerHeight(), establece u obtiene el alto de la ventana, sin incluir barras de herramientas.
- outerWidth(), establece u obtiene el ancho de la ventana, incluyendo las barras de herramientas.
- outerHeight(), establece u obtiene el alto de la ventana, incluyendo las barras de herramientas.

El método width() establece o devuelve el ancho de un elemento (excluye relleno, borde y margen).

El método height() establece o devuelve la altura de un elemento (excluye relleno, borde y margen).

El siguiente ejemplo devuelve el ancho y alto de un elemento especificado:

```
$("#boton").on("click", function(){
    lettxt = "";
    txt += "Ancho: " + $("#div1").width() + "</br>";
    txt += "Alto: " + $("#div1").height();
    $("#div1").html(txt);
});
```

El método innerWidth() devuelve el ancho de un elemento (incluye relleno).

El método innerHeight() devuelve la altura de un elemento (incluye relleno).

El siguiente ejemplo devuelve el ancho / alto interno de un elemento especificado:

```
$("#boton").on("click", function(){
    lettxt = "";
    txt += "Ancho interno: " + $("#div1").innerWidth() + "</br>";
    txt += "Alto interno: " + $("#div1").innerHeight();
    $("#div1").html(txt);
});
```

El método outerWidth() devuelve el ancho de un elemento (incluye relleno y borde).

El método outerHeight() devuelve la altura de un elemento (incluye relleno y borde).

El siguiente ejemplo devuelve el ancho / alto externo de un elemento especificado:

```
$("#boton").on("click", function(){
    lettxt = "";
    txt += "Ancho externo: " + $("#div1").outerWidth() + "</br>";
    txt += "Alto externo: " + $("#div1").outerHeight();
    $("#div1").html(txt);
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

```
});
```

El método outerWidth(true) devuelve el ancho de un elemento (incluye relleno, borde y margen).

El método outerHeight(true) devuelve la altura de un elemento (incluye relleno, borde y margen).

```
$("#boton").on("click", function(){
    lettxt = "";
    txt += "Ancho externo (+margen): " + $("#div1").outerWidth(true) + "</br>";
    txt += "Alto externo (+margen): " + $("#div1").outerHeight(true);
    $("#div1").html(txt);
});
```

El siguiente ejemplo devuelve el ancho y alto del documento (el documento HTML) y la ventana (la ventana del navegador):

```
$("#boton").on("click", function(){
    lettxt = "";
    txt += "Ancho/Alto del documento: " + $(document).width();
    txt += "x" + $(document).height() + "\n";
    txt += " Ancho/Alto de la ventana: " + $(window).width();
    txt += "x" + $(window).height();
    alert(txt);
});
```

El siguiente ejemplo establece el ancho y alto de un elemento especificado:

```
$("#boton").on("click", function(){
    $("#div1").width(500).height(500);
});
```

5.12 AlertifyJS

AlertifyJS es una librearía JavaScript que nos ayudará a incorporar a nuestras webs y aplicaciones diálogos y notificaciones potentes e impactantes.

En su página oficial podemos encontrar toda la información, multitud de ejemplos, las descargas y lo necesario para poder trabajar con esta fantástica librería.

Integración

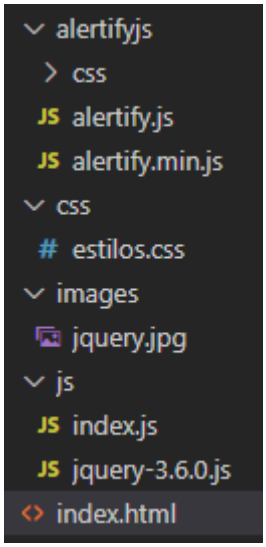
Igual que ocurre con JQuery, para poder usar esta librería tenemos la opción de conectar a un CDN, con las ventajas y desventajas ya conocidas, así como descargar la librería entera en nuestro equipo de desarrollo y/o producción y usarla directamente.

Para este taller se ha optado por la segunda opción, pudiendo descargar la librería desde la plataforma.

Una vez descargada, debemos descomprimirla; si echamos un vistazo al interior, podemos ver que no solo hay ficheros JavaScript, sino también ficheros de hojas de estilo. Depende de la que usemos tendremos una apariencia u otra. En los primeros ejemplos trataremos el tema y luego ya siempre usaremos uno de ellos.

IMPORTANTE: Para el correcto funcionamiento de AlertifyJS es imprescindible JQuery. Es decir, que en todos nuestros proyectos necesitamos las dos librerías, además de otros estilos, otros ficheros JavaScript, imágenes, ...

Nuestro proyecto podría tener un aspecto tal como se muestra en la imagen adjunta.



De momento comenzaremos con un ejemplo para ver la integración y cómo funciona de forma básica mediante un script en un fichero externo, similar a los vistos en el Taller de JQuery.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Veamos un sencillo ejemplo con lo mínimo:

```
<!DOCTYPEhtml>
<htmllang="es">

<head>
    <metacharset="UTF-8">
    <metahttp-equiv="X-UA-Compatible"content="IE=edge">
    <metaname="viewport"content="width=device-width, initial-scale=1.0">
    <linkrel="stylesheet"href="/alertifyjs/css/alertify.min.css"/>
    <title>JavaScript: JQuery</title>
</head>

<body>
    <h1>Taller AlertifyJS</h1>
    <buttonid="boton">Pulsar...</button>

    <!-- JavaScript -->
    <scriptlanguage="javascript"type="text/javascript"src=".\js\jquery-
3.6.0.js"></script>
    <scriptlanguage="javascript"type="text/javascript"src=".\js\index.js"></script>
    <scriptsrc="/alertifyjs/alertify.js"></script>
</body>

</html>
```

index.html

La integración de AlertifyJS implica la inclusión de una hoja de estilo en la cabecera...

```
<linkrel="stylesheet"href="/alertifyjs/css/alertify.min.css"/>
```

...y de la librería:

```
<scriptsrc="/alertifyjs/alertify.js"></script>
```

```
$("#boton").on("click", function(){
    alertify.alert('Tachán!');
});
```

index.js

Si lo probamos:

www.grupostudium.com
informacion@grupostudium.com
954 539 952

NOTA: El aspecto puede variar según el navegador donde se ejecute.

Probemos con otro estilo simplemente añadiendo la siguiente línea en la cabecera tras la del estilo de AlertifyJS que ya está, la básica:

```
<linkrel="stylesheet"href="/alertifyjs/css/themes/bootstrap.css"/>
```

Un poco más refinado el diseño. Este será el que usemos en el resto de los ejemplos.

Ejemplos

AlertifyJS nos permitirá trabajar con diálogos para mensajes (alert), para confirmar (confirm), para solicitar información (prompt) y notificaciones en la parte inferior del navegador (notifier).

Componente alert

Se usa para mostrar mensajes de información, de feedback, de tareas completadas, e incluso para mostrar unas notas de licencia, por ejemplo. las posibilidades son infinitas.

A menos que se indique lo contrario, continuamos con el fichero index.html.

El siguiente ejemplo permite también establecer el título del mensaje:

```
$("#boton").on("click", function(){
    alertify.alert('Alert Title', 'Alert Message!');
});
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

También podemos establecer el mensaje del botón:

```
$("#boton").on("click", function(){
    alertify.alert('Label changed!').set('label', 'Alright!');
});
```

Ahora mostraremos un mensaje sin botón de "OK", pero que se puede maximizar y minimizar:

```
$("#boton").on("click", function(){
    alertify.minimalDialog || alertify.dialog('minimalDialog',function(){
        return {
            main:function(content){
                this.setContent(content);
            }
        };
    });
    alertify.minimalDialog("Minimal button-less dialog.");
});
```

Se puede establecer si el diálogo de mensaje se pueda mover o no cuando aparece:

```
$("#boton").on("click", function(){
    alertify.alert('Movable: true').set('movable', true);
});
```

Y podemos delimitar a la pantalla del navegador por dónde mover el diálogo:

```
$("#boton").on("click", function(){
    alertify.alert('Try moving this off the screen').set({'movable':true,
'moveBounded':true});
});
```

Por último, podemos establecer el tipo de transición cuando aparezca, es decir, el efecto animado de cómo aparece el mensaje:

```
$("#boton").on("click", function(){
    alertify.dialog('alert').set({transition:'flipy',message:'Transition
effect: flipy'}).show();
});
```

En este caso, podemos cambiar "flipy" en la propiedad "transition:" por "slide", "zoom", flipx", "fade" y "pulse".

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Componente notifier

Este componente se usa exclusivamente para mostrar un breve mensaje unos instantes y luego, automáticamente desaparece.

El caso más básico es:

```
$("#boton").on("click", function(){
    alertify.notify('Mensajito', 5);
});
```

El número 5 indica el número de segundos que está visible el mensajito antes de ocultarse automáticamente. Dicho valor es el que se toma por defecto en caso de no especificar ninguno. Si se indica un 0, estará visible hasta pulsarlo.

Por defecto, el mensaje aparece abajo a la derecha. Pero esta posición se puede establecer mediante un método set(). Tenemos seis posibilidades: arriba a la izquierda, arriba en el centro, arriba a la derecha, abajo a la izquierda, abajo en el centro y abajo a la derecha. El siguiente ejemplo lo mostrará arriba en el centro:

```
$("#boton").on("click", function(){
    alertify.set('notifier','position', 'top-center');
    alertify.notify('Mensajito');
});
```

A parte del método notify(), existen otros métodos para expresar mediante colores, otros tipos de mensajes: el método success() se asimila al éxito, al color verde; el método error() rojo para el error y el método warning()el amarillo para una advertencia.

Éxito:

```
$("#boton").on("click", function(){
    alertify.sucess('Mensajito');
});
```

Error:

```
$("#boton").on("click", function(){
    alertify.error('Mensajito');
});
```

Advertencia:

```
$("#boton").on("click", function(){
    alertify.warning('Mensajito');
});
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

Un último ejemplo curioso que muestra un mensaje cuyo contenido cambia con el paso del tiempo:

```
$("#boton").on("click", function(){
    letduration = 10;
    letmsg = alertify.message('Auto-dismiss in '+ duration +' seconds.', 10,
function(){ clearInterval(interval);});
    letinterval = setInterval(function(){
        msg.setContent('Auto-dismiss in '+(--duration)+' seconds.');
 },1000);
});
```

Componente confirm

Este componente permite no solo mostrar un mensaje si no que le ofrece al usuario la posibilidad de elegir varias opciones según los botones que se mostrarán:

```
$("#boton").on("click", function(){
    alertify.confirm('Confirm Title', 'Confirm Message',
    function(){ alertify.success('Ok') },
    function(){ alertify.error('Cancel')}
    );
});
```

En este primer ejemplo, simplemente mostramos los botones por defecto que, al pulsarlos, se ejecutarán las funciones indicadas, una notificación de éxito para el "OK", una notificación de error para el "Cancel".

Mediante el método set() y la propiedad labels podemos establecer el mensaje de los botones. En el siguiente ejemplo se especificará "Sí" y "No":

```
$("#boton").on("click", function(){
    alertify.confirm('Etiquetas personalizadas').set('labels', {ok:'Sí', cancel:'No'});
});
```

Componente prompt

Por último, tenemos un cuadro de diálogo en el que el usuario puede hacer algo más que pulsar unos simples botones. Estamos hablando de un diálogo que permite escribir en una zona detexto.

En el siguiente ejemplo se mostrará un diálogo con un cuadro de texto donde escribir. Lo que se escriba será mostrado en una notificación al pulsar el botón de "OK". Si se pulsa "Cancel" se mostrará simplemente una notificación de error:

```
$("#boton").on("click", function(){
    alertify.prompt( 'Prompt Title', 'Prompt Message', 'Prompt Value'
                , function(evt, value) { alertify.success('You entered: ' + value) }
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

```
                , function() { alertify.error('Cancel') });
});
```

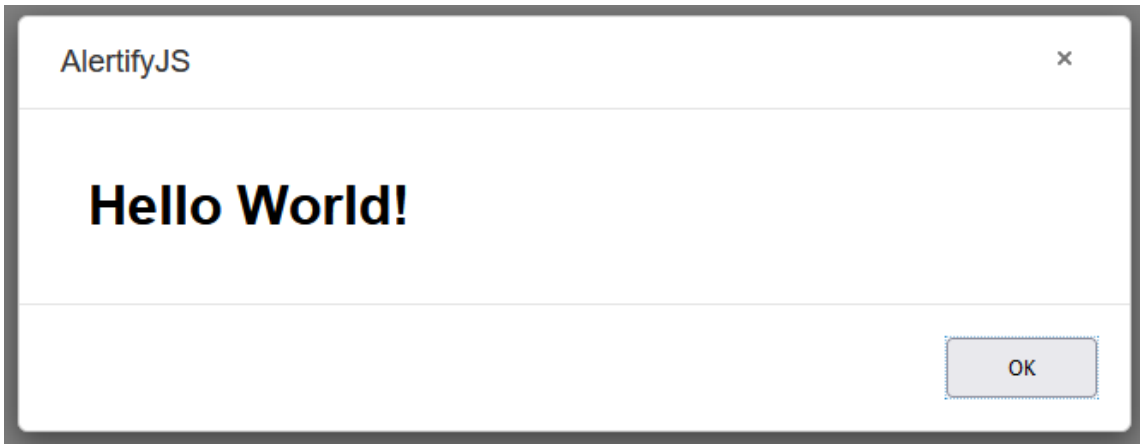
Igualmente se pueden personalizar los botones con set() y labels.

Método setContent()

En las notificaciones hemos visto un ejemplo en el que podíamos establecer un comportamiento especial para el contenido de manera que se modificaba el mensaje mostrado gracias al método setContent().

Realmente este método se puede usar con cualquiera de los elementos anteriores. Por ejemplo, podemos meter código HTML dentro de una alerta:

```
$("#boton").on("click", function(){
    alertify.alert().setContent('<h1> Hello World! </h1>').show();
});
```



Pero cuidado con este método pues sustituye su contenido por TODO el contenido del método usado: alert(), prompt(), confirm() y las diferentes notificaciones, como ya vimos en su momento.

5.13 Depuración de código

Una de las mejores herramientas que puede (y debe) usar cualquier programador, ya sea de aplicaciones de escritorio y web, son los **depuradores de código**.

En muchos IDE y Frameworks vienen integrados en el propio programa que se usa para programar. En el caso de desarrollo de aplicaciones web, los propios navegadores han ido dotándose de unas herramientas que ayudan muchísimo a la hora de depurar nuestro trabajo destacando muchos detalles con precisión sobre

www.grupostudium.com
informacion@grupostudium.com
954 539 952

la propia página web con lo que podemos ver el comportamiento de ciertos elementos o cómo va quedando nuestra web. Y no sólo en su aspecto exterior del diseño en HTML y CSS, sino también en la programación en JavaScript (o cualquier otro lenguaje de script).

Independientemente del navegador que estemos usando para "ver" nuestros trabajos, dichas herramientas están disponibles pulsando la tecla de función F12.

Sin embargo, cada navegador ha desarrollado sus propias opciones, comunes en algunos casos, únicas en otros. Nosotros estudiaremos en profundidad las de **Chrome** por ser el más estándar con respecto a HTML, CSS y JavaScript. Aunque también destacaremos algunas otras opciones que incorporan los otros navegadores.

Google Chrome

Estudiaremos todas las opciones de este navegador que tenemos disponibles al pulsar la tecla de función F12.

Las opciones vienen en una barra de menú con dos partes diferenciadas a izquierda y derecha.

En la parte inferior aparece la información de lo que hayamos elegido en las opciones superiores.

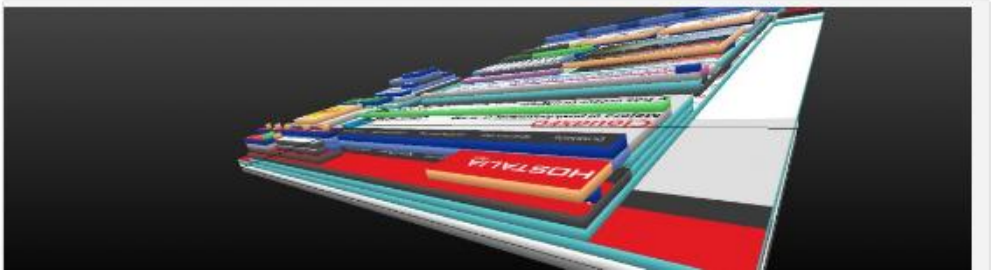
Comenzaremos de izquierda a derecha. Lo primero que encontramos es una herramienta para poder seleccionar elementos que aparecen en la parte superior, donde aparece la web que estamos analizando.

Mediante esta herramienta podemos desplazarnos por la web y se nos irán marcando los diferentes elementos que aparecen con una mínima información

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Como ya sabrás puedes acceder al html de cualquier web pulsando con botón derecho en la página y luego en 'Ver código fuente', pero para editarlo sigue los pasos que te explica a continuación el Dr. Hosting. Las funcionalidades varían según el navegador que uses, pero básicamente el procedimiento es el mismo.

p | 750 × 95



Si pulsamos sobre uno de ellos, en la parte inferior aparecerá dicho elemento destacado entre el código HTML, así como su estilo que tiene aplicado, como se indica en la parte derecha.

La siguiente herramienta nos permite "cambiar" nuestro dispositivo de visualización, simulando que estamos en una Tablet, un teléfono móvil, etc. Podemos configurar diferentes tamaños de pantalla y ver si nuestro diseño es Responsive o no.

Se puede observar que si cambiamos algo, tanto en HTML como en CSS, automáticamente vamos a ver los efectos en la web de arriba. Obviamente estos cambios son temporales, no estamos cambiando la web realmente, solamente su "visualización" de forma provisional para ver cómo quedan ciertos cambios y si es lo que buscamos, luego habrá que cambiar el fichero original.

Inicialmente, al entrar en estas herramientas de depuración, la opción marcada por defecto es "**Elements**", que se compone de una pantalla en la parte inferior con el código HTML, una pantalla en la parte inferior derecha con los estilos CSS aplicados al elemento destacado, y otra ventana abajo del todo, la "**Console**" con diferentes mensajes del navegador, sobre todo advertencias, errores y problemas de estilos y de scripts.

La ventana de estilos se divide a su vez en:

- "Styles", para ver los estilos aplicados, los ficheros que contienen dichos estilos y las líneas de los ficheros donde se describe el estilo
- "Computed", donde podemos analizar la estructura de Cajas
- "Event Listeners", para ver los eventos que se producen en nuestra web

www.grupostudium.com
informacion@grupostudium.com
954 539 952

- "DOM breakpoitns" para estudiar los puntos de espera que podemos marcar en nuestro DOM
- "Properties", para estudiar la estructura completa de la web

La siguiente opción del menú es "**Console**", donde podemos ver una detallada muestra de errores, advertencias e incluso donde podremos escribir sentencias de script para probar ciertos comportamientos y ciertas instrucciones de JavaScript, por ejemplo.

En "**Sources**" podemos cargar cualquier fichero disponible en el actual sitio para su estudio detallado. Ahora podremos trabajar con una ventana que aparece en la parte derecha que nos permitirá crear "visores" y establecer puntos de parada para hacer un seguimiento línea a línea si hace falta de nuestro código de script. También se puede hacer con HTML y CSS, pero su uso es más limitado.

En "**Network**" se nos muestra información sobre cada elemento de la página que se visualiza en la parte superior. Propiedades como nombre, método usado para la carga, código de estado devuelto pro el servidor, tipo de elemento, tamaño, tiempo que ha tardado en cargar, en qué lugar se ha iniciado y acabado su carga con respecto a los demás, ... Este apartado está especialmente pensado para hacer un estudio de rendimiento de la carga de la página web, por si tarda mucho, ver qué elementos son los que hacen que se ralentice dicha carga y trabajar sobre ellos: imágenes grandes, scripts que demoran mucho su desarrollo, etc.

Name	Method	Status	Type	Initiator	Size	Time	Waterfall	4.00 s
explorer-blog-hostalia-hosting.jpg	GET	200	jpeg	Other	41.5 KB	85 ms		
firefox-blog-hostalia-hosting.jpg	GET	200	jpeg	Other	33.6 KB	83 ms		
opera-blog-hostalia-hosting.jpg	GET	200	jpeg	Other	42.0 KB	130 ms		
triangulo-editor-html-blog-hostalia-hosting.jpg	GET	200	jpeg	Other	104 KB	194 ms		
cambiar-navegador-blog-hostalia-hosting.jpg	GET	200	jpeg	Other	86.1 KB	94 ms		
editor-html-blog-hostalia-hosting.jpg	GET	200	jpeg	Other	62.8 KB	102 ms		

33 requests | 1.9 MB transferred

"**Performance**" permite grabar una secuencia de carga, de interacción con la web o cualquier otra operación que se pueda hacer en la web para un posterior estudio de los elementos que interaccionen entre sí, los tiempos que tardan en realizar sus tareas, quién espera a quién, etc. También esta opción está encaminada a estudios de rendimiento y posterior mejora.

La opción "**Memory**" permite hacer una captura de cuánta memoria están ocupando los elementos cargados de la web estudiada para su posterior análisis y mejorar el rendimiento de la página.

En la siguiente opción, "**Application**" podemos encontrar todo tipo de herramientas y propiedades en la ventana que aparece en la parte izquierda relativos a accesos a bases de datos, cookies, datos guardados en caché, etc.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Los recursos que aquí se nos ofrecen sirven para la adecuada configuración de aplicaciones web que requieran de interacción con datos tanto en los servidores como en los propios navegadores.

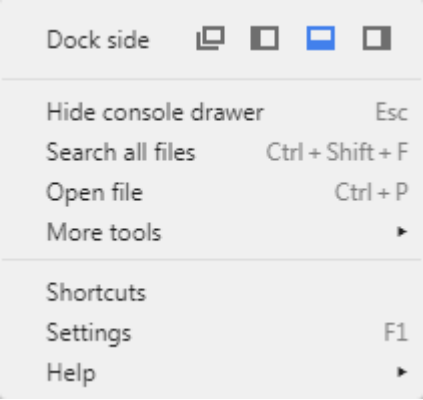
Como seguridad, en "**Security**" podemos encontrar información importante acerca de certificados digitales, firmas electrónicas y todo lo que conlleva la transmisión segura de datos y la identificación correcta de los extremos en una comunicación cliente-servidor.

Por último, en "**Lighthouse**" podemos realizar toda una auditoría de nuestra web realizada por Chrome para obtener un informe con detección de errores, cómo corregirlos, cómo mejora el rendimiento, cómo hacer responsive nuestra web, etc.

Muy completo y gratuito.

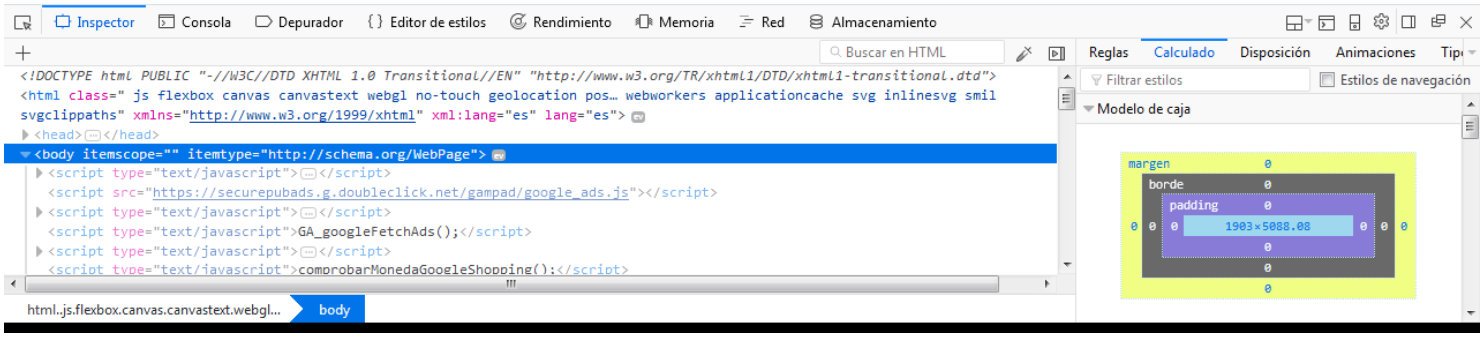
No debemos olvidar de señalar los errores y advertencias que se nos muestran en la barra de menú, pero en la parte derecha.

Y finalmente tenemos disponible en la herramienta de los botones, varias opciones sobre la propia herramienta: Organización de las ventanas, ayudas, otras opciones más, configuración de la propia herramienta, etc.



Mozilla Firefox

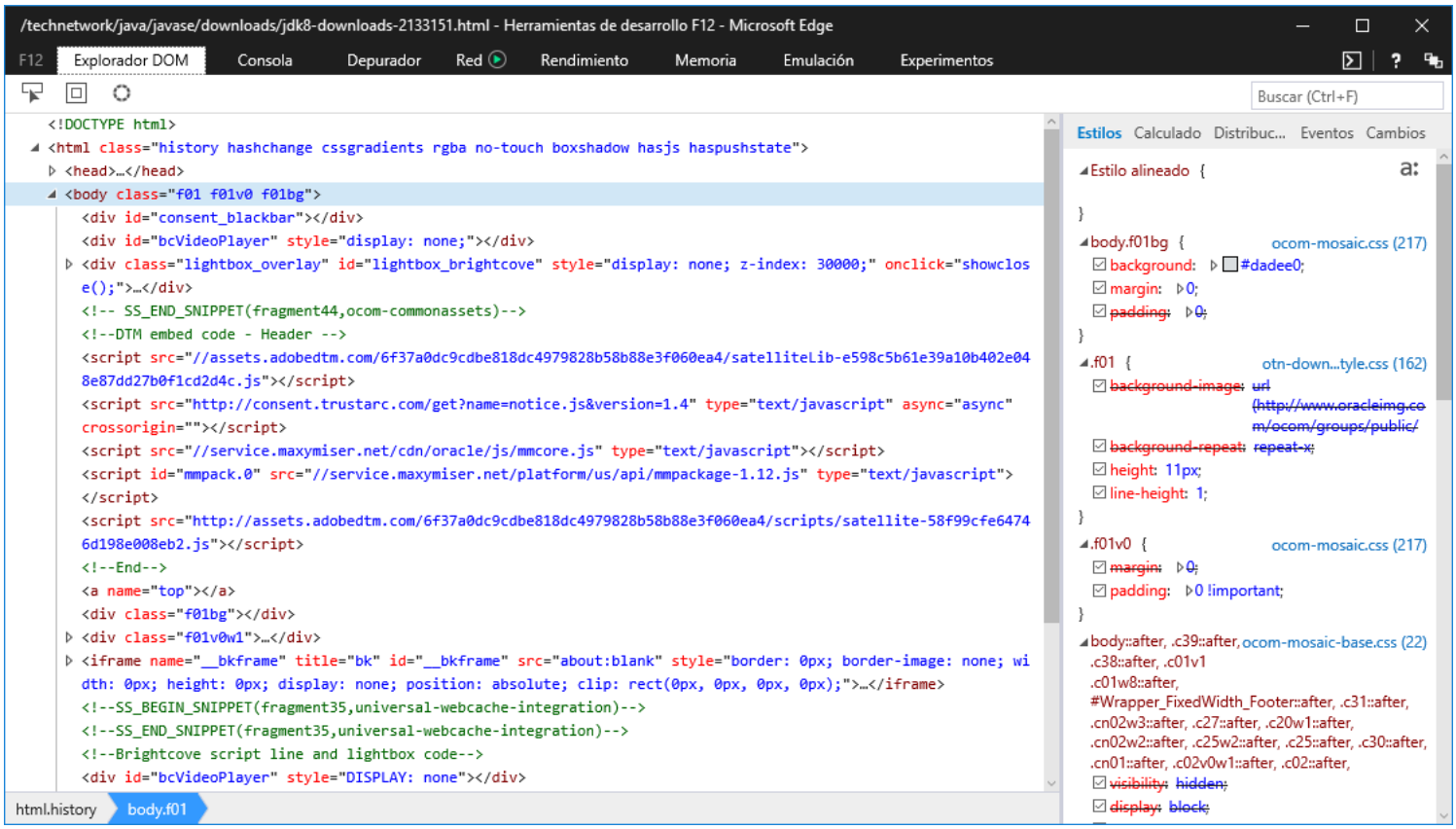
Aquí vamos a encontrar las mismas opcione,s pero con distintos nombres.



El selector, "Inspector", que sería la opción por defecto, "Consola", "Depurador", "Editor de estilos", "Rendimiento", "Memoria", "Red" y "Almacenamiento".

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Microsoft Edge



En el nuevo navegador de Microsoft, también tenemos estas herramientas, con las opciones de “Explorador DOM”, “Consola”, “Depurador”, “Red”, “Rendimiento”, “Memoria”, “Emulación” y “Experimentos”.

08/07/2025

www.grupostudium.com
informacion@grupostudium.com
954 539 952